

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DataBase Management System

PROJECT REPORT

InHabit - Hostel Grievance Management System

Name: Raghav Sarda & Aniruddha Pratap Singh

PRN: 25070122**235** & 25070122**247**

Batch: CSE – C:**C2**

Faculty: Dr. Deepali Vora, Dr. Balkrishna Patil

SEMESTER - 03

A.Y. 2026-2027

TABLE OF CONTENTS

1. Scope Of The System.....4
1.1 Problem Context.....4
1.2 Users and Characteristics.....4
1.3 In-Scope Features.....5
1.4 Out-of-Scope Features.....5

2. Requirements.....5
2.1 Functional Requirements.....5
2.2 Non-Functional Requirements.....6

3. Entity-Relationship (ER) Modeling.....8
3.1 Entity Classification.....8
3.2 Attribute Classification & Constraints.....9
3.3 Structural Relationships & Cardinalities.....10

4. Relational Database Schema.....11

5. Extended ER (EER) Specialization & Generalization.....14
5.1 Disjoint & Total Specialization of USER (d, t).....14
5.2 Disjoint Specialization of STAFF (d).....15
5.3 Specialization of COMPLAINT.....15

6. Identification of Database Anomalies.....15
6.1 1NF Anomalies.....16
6.2 2NF Anomalies.....16
6.3 3NF Anomalies.....17

7. Functional Dependencies.....18

8. Database Normalization.....19
8.1 First Normal Form (1NF).....20
8.2 Second Normal Form (2NF).....20
8.3 Third Normal Form (3NF).....21

8.4 Boyce-Codd Normal Form (BCNF) Verification.....22

9. Application of Codd's Rules.....22

10. Implementation of Tables in MySQL.....24

10.1 MySQL DDL Implementation.....24

10.2 Table Outputs (Sample Verification).....27

11. Queries, Functions, Procedures & Triggers Implemented.....30

11.1 Analytical Queries Implemented.....30

11.2 View Implementation.....32

11.3 Function Implementation.....32

11.4 Stored Procedure Implementation.....33

11.5 Trigger Implementation.....34

12. Project Demonstration.....35

13. References & Tools Used.....44

Contributing Members.....45

GitHub Link: https://github.com/iamthersforever/InHabit

---

1. Scope Of The System

InHabit is a web-based hostel grievance (complaint and ticketing) management system, designed to provide a transparent, centralized, and responsible platform where hostel residents can report, monitor, and solve hostel-related problems.
The system replaces the traditional manual and informal complaint-handling with a formal structured and auditable ticketing process, allowing community-based prioritization and clear accountability for the hostel authorities.

1.1 Problem Context

Poor maintenance, food quality, internet connectivity, cleanliness, water provision, and security are commonly reported problems, usually reported through informal verbal communication or through manual registers.

Such actions frequently result in:

- Delayed issue resolution.
- Lack of accountability/responsibility.
- Frequent/Repeated complaints.
- No historical and analytical information.
- InHabit solves these issues by converting informal complaints into traceable digital tickets with defined processes and status visibility, thereby enhancing work efficiency, transparency, and accountability in managing issues in the hostels.

1.2 Users and Characteristics

USER ROLE	DESCRIPTION
Students	Post/Submit complaints, cast vote on issues and view/track resolution status
Hostel Authorities / Wardens	Review, prioritize, update and close complaints
Maintenance Staff	Resolve assigned complaints and update progress
System Administrators	Administer user roles, moderation rules/policies, and system settings.
Academic Reviewers	View aggregated system reports (read-only)

1.3 In-Scope Features

- Authenticated and secure login using institutional college email.
- Creation of complaints with category tags (e.g. Mess, Internet, Water, Cleanliness, Security) and optional media attachments.
- Community-based reviewed prioritization of complaints using "Genuine" and "Mid" voting.
- Automatic escalation of complaints – complaints with more than ‘30 Genuine Votes’ are labelled as ‘Concerning’.
- Complaint status tracking with states:
- Open → Under Review → In Progress → Resolved → Closed
- Role-based authority reviewing and complaint closing.
- A "What’s Happening" tab for college, hostel, academic, and student-related updates.
- Lost and Found section for hostel residents.

1.4 Out-of-Scope Features

- Emergency Contact Information
- Email-based notification system to apprise users about the complaint escalations and status updates.
- Monthly feedback system for future improvements and addition of features.
- Generation of analytical reports for hostel authorities and audit committees.
- Emergency response systems
- Automated moderation and student restriction mechanisms to prevent misuse.

---

2. Requirements

2.1 Functional Requirements

1. *Authentication:* The system shall authenticate users through official college email (college domain only).
2. *Complaint Submission:* The system shall allow the students to submit categorized complaints (text complaints ≤ 200 words) with a maximum of 10 attachments (images/video; files ≤ 100 MB each).

3. *Voting & Prioritization:* The system shall allow upvoting by the community; complaints that have votes above a certain threshold (30) shall be labelled as *Concerning*.
4. *System Escalation:* The system shall automatically escalate complaints marked as *Concerning* or those match the escalation criteria (e.g., votes + time unresolved).
5. *Status Tracking:* The system shall enable the definition of statuses for the tickets (Open, Under Review, In Progress, Resolved) and show corresponding timestamps and history.
6. *Ticket Closure:* The system shall require a confirmation from both the assigned authority (warden/technician) and complainant before closing a ticket for sensitive or escalated complaints.
7. *Notifications:* The system shall alert/notify (email) for important events: ticket creation, assignment, status changes, escalation, and closure.
8. *Moderation & Spam Control:* The system shall enable the detection/highlighting of abusive language, prevent repeated/duplicate complaints, and enable administrators to suspend the repeat offenders.
9. *Search & Filtering:* The system shall enable search and filters by category, status, date, votes, and tags.
10. *Reports & Analytics:* The system shall enable the generation of administrative summary reports (e.g., monthly counts, average time for resolution of the tickets, trending issues).

2.2 Non-Functional Requirements

Performance:

- Standard UI interactions should take ≤ 2 seconds on a typical campus internet connection.
- Image upload should display progress and compress locally (on-device) to minimize bandwidth.

Security & Privacy:

- Role-based access control (student, warden/admin, maintenance staff) must be implemented.

- All network communications shall use HTTPS; sensitive fields must be encrypted at rest (e.g., student contact information for sensitive complaints).
- Sensitive complaints should be anonymous in public views unless admin authorization is provided.

Usability:

- Mobile-first design for students; desktop-optimized dashboard for wardens.
- Primary actions/Common tasks (file complaint, upvote, update status) should be accessible in ≤ 2 taps/clicks from the home screen.

Reliability & Availability:

- Complaint data should not be lost in case of server failure – implement periodic database backups.

Scalability:

- Campus-wide support (thousands of students) and scaling to multiple hostels without significant/major redesign.

Maintainability:

- Clean and modular codebase and proper documentation.

Legal & Compliance:

- Adhere to/Comply with the institutional data-retention and privacy policies.

---

3. Entity-Relationship (ER) Modeling

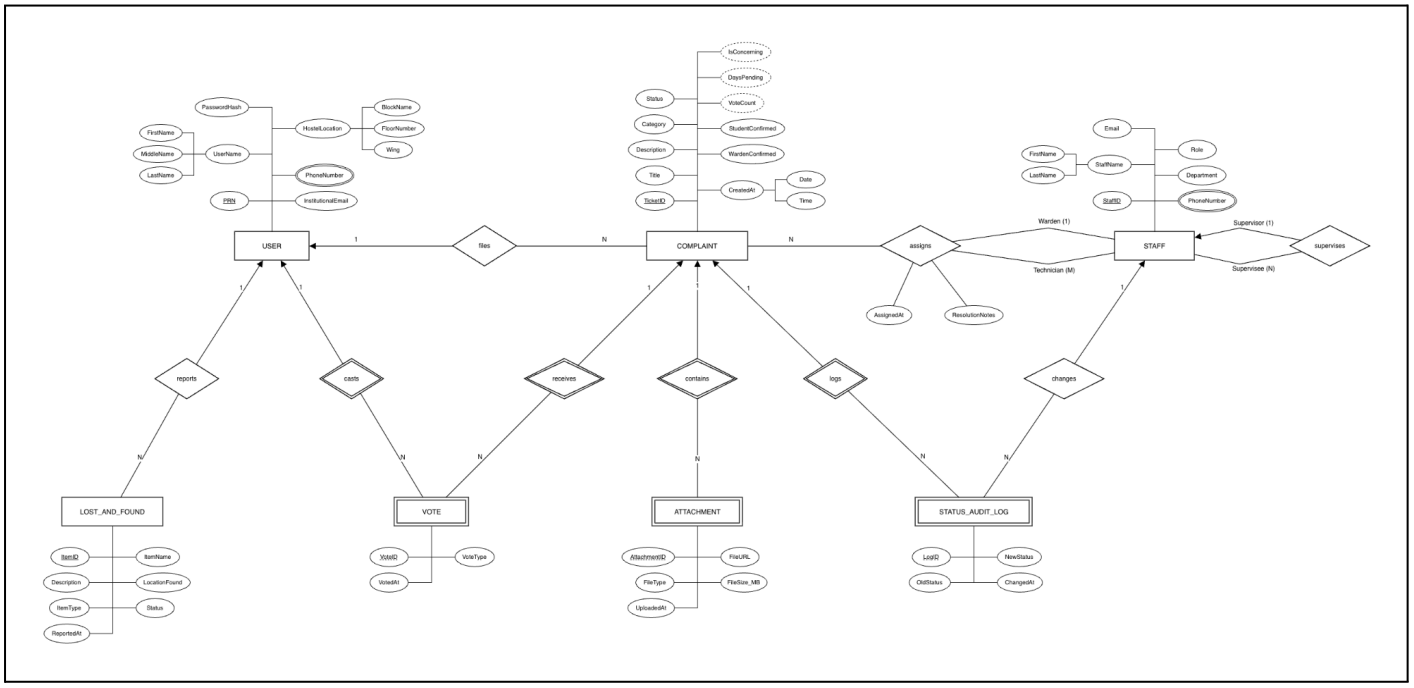


Fig: ER Diagram for InHabit Hostel Grievance Management System

3.1 Entity Classification

Strong Entities (Independent Existence)

- USER: Central entity representing all registered accounts in the system, uniquely identified by PRN (in conceptual ER) / UserID (in relational model).
- STAFF: Dedicated entity representing hostel administrative authorities and technical personnel, uniquely identified by StaffID.
- COMPLAINT: Core operational entity storing grievance tickets submitted by residents, uniquely identified by TicketID.
- LOST_AND_FOUND: Independent entity managing lost and found campus listings, uniquely identified by ItemID.

Weak Entities (Existence-Dependent)

- VOTE: Represents community prioritization votes ("Genuine" / "Mid"). It is existence-dependent on both USER and COMPLAINT, identified by the partial discriminator / key VoteID.
- ATTACHMENT: Represents physical evidence files uploaded for a complaint, existence-dependent on COMPLAINT and identified by AttachmentID.

- STATUS_AUDIT_LOG: Represents immutable chronological state transition records, existence-dependent on COMPLAINT and identified by LogID.

3.2 Attribute Classification & Constraints

1. Key Attributes

- PRN / UserID in USER
- StaffID in STAFF
- TicketID in COMPLAINT
- VoteID in VOTE
- AttachmentID in ATTACHMENT
- LogID in STATUS_AUDIT_LOG
- ItemID in LOST_AND_FOUND

2. Composite Attributes

- UserName: Branched into FirstName, MiddleName, and LastName.
- StaffName: Branched into FirstName and LastName.
- HostelLocation: Branched into BlockName, FloorNumber, and Wing.
- CreatedAt: Branched into Date and Time.

3. Simple / Single-Valued Attributes

- In USER: PasswordHash, InstitutionalEmail.
- In STAFF: Email, Department, Role.
- In COMPLAINT: Title, Description, Category, Status, WardenConfirmed, StudentConfirmed.
- In VOTE: VoteType, VotedAt.
- In ATTACHMENT: FileURL, FileType, FileSize_MB, UploadedAt.
- In STATUS_AUDIT_LOG: OldStatus, NewStatus, ChangedAt.
- In LOST_AND_FOUND: ItemName, Description, LocationFound, ItemType, Status, ReportedAt.

4. Multi-Valued Attributes

- PhoneNumber in USER: Residents may have multiple contact numbers (normalized to USER_PHONE in relational design).

- PhoneNumber in STAFF: Staff members may maintain secondary contact numbers.

5. Derived Attributes

- VoteCount: Dynamically tallied from total genuine vote records linked to the complaint.
- DaysPending: Computed dynamically as CURRENT_DATE - CreatedAt.
- IsConcerning: Derived boolean flag set automatically when VoteCount >= 30.

3.3 Structural Relationships & Cardinalities

1. Unary Relationship

- STAFF *supervises* STAFF (1:N): A supervisory authority (Warden / Supervisor, cardinality 1) manages and oversees multiple maintenance staff members (Technicians, cardinality N).

2. Binary Relationships

- USER *files* COMPLAINT (1:N): One student can file zero, one, or many complaints (N); each complaint is logged by exactly one student (1).
- USER *casts* VOTE (1:N): One student can cast multiple votes across tickets (N); each vote belongs to one student (1).
- COMPLAINT *receives* VOTE (1:N): One complaint can accumulate multiple community votes (N).
- COMPLAINT *contains* ATTACHMENT (1:N): One complaint contains zero to many evidence files (N).
- COMPLAINT *logs* STATUS_AUDIT_LOG (1:N): One complaint generates multiple state transition logs (N).
- STAFF *changes* STATUS_AUDIT_LOG (1:N): One staff member can execute multiple status change actions recorded in the log (N).
- USER *reports* LOST_AND_FOUND (1:N): One student can post zero, one, or multiple lost/found listings (N).

3. Ternary Relationship

- STAFF (Warden) *assigns* STAFF (Technician) to COMPLAINT: A single ternary association (assigns) linking:
    - STAFF acting as Warden (cardinality 1)
    - STAFF acting as Technician (cardinality M)
    - Target COMPLAINT (cardinality N)
    - Carries relationship attributes: AssignedAt and ResolutionNotes.

---

4. Relational Database Schema

1. USER

- USER (UserID, FirstName, MiddleName, LastName, InstitutionalEmail, PasswordHash, Role)
- Primary Key: UserID
- Alternate Key / Unique: InstitutionalEmail

2. USER_PHONE *(Normalized Multi-Valued Attribute)*

- USER_PHONE (PhoneID, *UserID*, PhoneNumber)
- Primary Key: PhoneID
- Foreign Key: UserID REFERENCES USER(UserID) ON DELETE CASCADE

3. STUDENT *(Subclass of USER)*

- STUDENT (UserID, PRN, BlockName, FloorNumber, Wing)
- Primary Key & Foreign Key: UserID REFERENCES USER(UserID) ON DELETE CASCADE
- Alternate Key / Unique: PRN

4. STAFF *(Subclass of USER)*

- STAFF (UserID, StaffID, Department)
- Primary Key & Foreign Key: UserID REFERENCES USER(UserID) ON DELETE CASCADE
- Alternate Key / Unique: StaffID

5. ADMIN *(Subclass of STAFF)*

- ADMIN (UserID, AccessLevel)

- Primary Key & Foreign Key: UserID REFERENCES STAFF(UserID) ON DELETE CASCADE

6. WARDEN *(Subclass of STAFF)*

- WARDEN (UserID, BlockInCharge)
- Primary Key & Foreign Key: UserID REFERENCES STAFF(UserID) ON DELETE CASCADE

7. TECHNICIAN *(Subclass of STAFF)*

- TECHNICIAN (UserID, Trade, *SupervisorID*)
- Primary Key & Foreign Key: UserID REFERENCES STAFF(UserID) ON DELETE CASCADE
- Foreign Key (Unary): SupervisorID REFERENCES WARDEN(UserID) ON DELETE SET NULL

8. COMPLAINT

- COMPLAINT (TicketID, Title, Description, Category, Status, CreatedAt, WardenConfirmed, StudentConfirmed, *FiledBy*)
- Primary Key: TicketID
- Foreign Key: FiledBy REFERENCES STUDENT(UserID) ON DELETE CASCADE

9. ESCALATED_COMPLAINT *(Subclass of COMPLAINT)*

- ESCALATED_COMPLAINT (TicketID, EscalatedAt)
- Primary Key & Foreign Key: TicketID REFERENCES COMPLAINT(TicketID) ON DELETE CASCADE

10. SENSITIVE_COMPLAINT *(Subclass of COMPLAINT)*

- SENSITIVE_COMPLAINT (TicketID, IsAnonymous)
- Primary Key & Foreign Key: TicketID REFERENCES COMPLAINT(TicketID) ON DELETE CASCADE

11. ATTACHMENT

- ATTACHMENT (AttachmentID, *TicketID*, FileURL, FileType, FileSize_MB, UploadedAt)
- Primary Key: AttachmentID
- Foreign Key: TicketID REFERENCES COMPLAINT(TicketID) ON DELETE CASCADE

12. VOTE

- VOTE (VoteID, *TicketID*, *UserID*, VoteType, VotedAt)
- Primary Key: VoteID
- Foreign Keys:
  - TicketID REFERENCES COMPLAINT(TicketID) ON DELETE CASCADE
  - UserID REFERENCES STUDENT(UserID) ON DELETE CASCADE
- Integrity Constraint: UNIQUE(TicketID, UserID) prevents duplicate voting.

13. STATUS_AUDIT_LOG

- STATUS_AUDIT_LOG (LogID, *TicketID*, *ChangedBy*, OldStatus, NewStatus, ChangedAt)
- Primary Key: LogID
- Foreign Keys:
  - TicketID REFERENCES COMPLAINT(TicketID) ON DELETE CASCADE
  - ChangedBy REFERENCES STAFF(UserID) ON DELETE SET NULL

14. TICKET_ASSIGNMENT *(Representation of Ternary Relationship)*

- TICKET_ASSIGNMENT (AssignmentID, *TicketID*, *WardenID*, *TechnicianID*, AssignedAt, ResolutionNotes)
- Primary Key: AssignmentID
- Foreign Keys:
  - TicketID REFERENCES COMPLAINT(TicketID) ON DELETE CASCADE
  - WardenID REFERENCES WARDEN(UserID) ON DELETE SET NULL
  - TechnicianID REFERENCES TECHNICIAN(UserID) ON DELETE SET NULL

15. LOST_AND_FOUND

- LOST_AND_FOUND (ItemID, ItemName, Description, LocationFound, ItemType, Status, ReportedAt, *ReportedBy*)
- Primary Key: ItemID
- Foreign Key: ReportedBy REFERENCES USER(UserID) ON DELETE SET NULL

---

5. Extended ER (EER) Specialization & Generalization

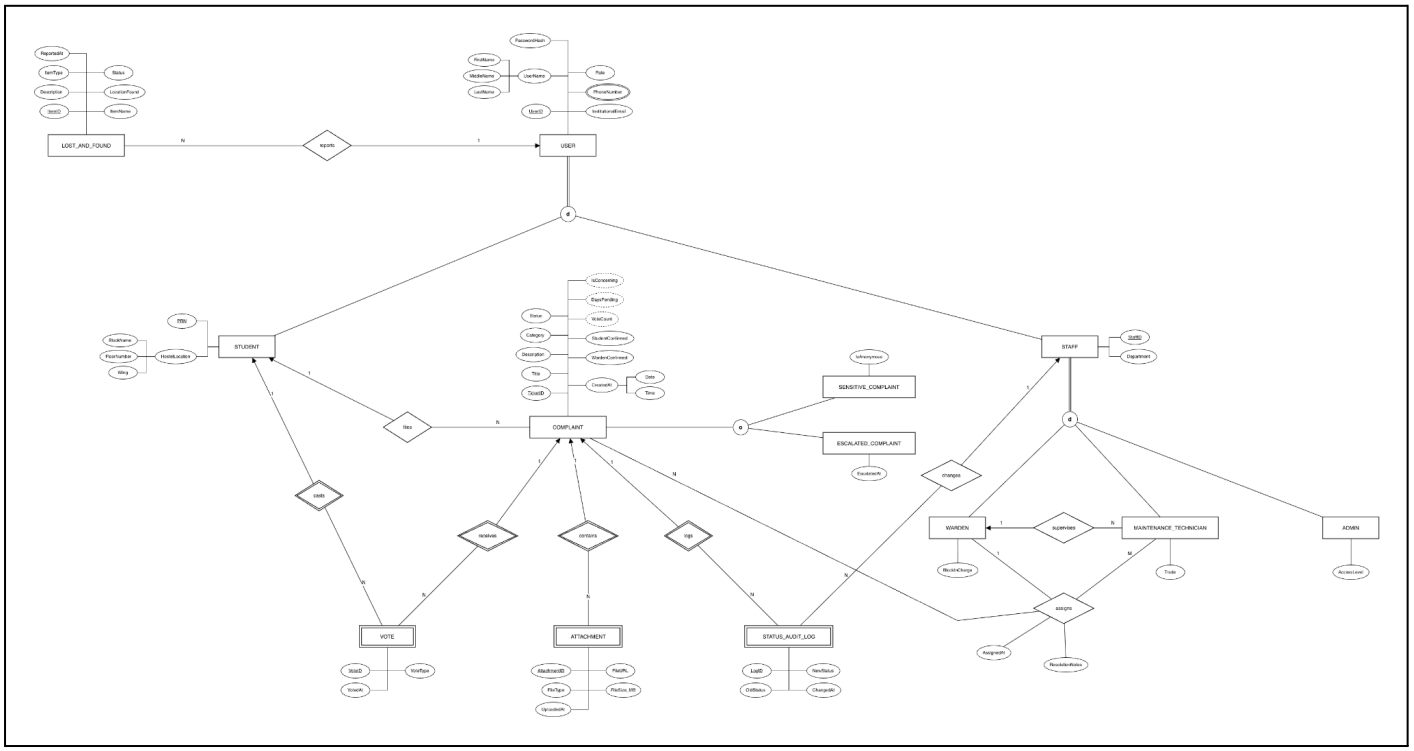


Fig: EER Diagram for InHabit Hostel Grievance Management System

As modeled in the EER diagram, object-oriented software methods (login(), register()) are omitted; data hierarchy is represented via formal EER constraints:

5.1 Disjoint & Total Specialization of USER (d, t)

- Superclass: USER (UserID, UserName, PasswordHash, PhoneNumber, InstitutionalEmail, Role).

- Specialization Circle: Disjoint (d) with double line (Total participation) — every registered account must be either a Student or Staff member.
- STUDENT Subclass: Inherits all USER attributes; introduces specific attributes: PRN (Unique) and composite HostelLocation (BlockName, FloorNumber, Wing).
- STAFF Subclass: Inherits all USER attributes; introduces specific attributes: StaffID (Unique) and Department.

5.2 Disjoint Specialization of STAFF (d)

- Superclass: STAFF
- Specialization Circle: Disjoint (d) categorizing operational personnel into three non-overlapping roles:
    - ADMIN Subclass: Introduces AccessLevel.
    - WARDEN Subclass: Introduces BlockInCharge (wing responsibility).
    - MAINTENANCE_TECHNICIAN Subclass: Introduces Trade (e.g., Electrical, Plumbing, Network).
- Recursive Hierarchy: Connected by binary relationship supervises where 1 WARDEN supervises N MAINTENANCE_TECHNICIAN personnel.

5.3 Specialization of COMPLAINT

- Superclass: COMPLAINT (TicketID, Title, Description, Category, Status, CreatedAt, WardenConfirmed, StudentConfirmed).
- Subclasses:
    - SENSITIVE_COMPLAINT: Introduces boolean IsAnonymous for masking student identification in public feeds.
    - ESCALATED_COMPLAINT: Introduces EscalatedAt timestamp recording when community engagement reached >= 30 genuine votes.

---

6. Identification of Database Anomalies

A database anomaly is a flaw within a database that occurs due to poor planning or when multi-themed data is stored in an unnormalized flat table. These anomalies lead to data redundancy and inconsistency.

6.1 1NF Anomalies

- *Insert Anomaly:* In an unnormalized flat table storing student phone numbers or complaint evidence files within a single cell, inserting a complaint with multiple attachments or multiple phone numbers violates atomicity (1NF). To insert multiple files, all other student and ticket attributes must be duplicated across rows.
- *Update Anomaly:* Updating a student's phone number requires updating multiple rows if that student has filed multiple complaints or uploaded multiple attachments, leading to potential data inconsistencies.
- *Delete Anomaly:* Deleting a specific evidence attachment stored in a multi-valued cell can result in losing information about the complaint or other attachments in the same record.

Unnormalized Table Violating 1NF:

TicketID	StudentPRN	StudentName	Room	PhoneNumbers	Category	EvidenceFiles
1001	257122235	Raghav Sarda	Alpha-101	9284883087, 9876122235	Internet	wifi_speedtest.png, trace.log
1002	257122247	Aniruddha Singh	Alpha-102	9838338899	Plumbing	valve_leak.jpg
1003	257122001	A Visvajit	Alpha-508	9876122001, 9123456780	Mess	turbid_water.jpg, food_sample.jpg

6.2 2NF Anomalies

- *Insert Anomaly:* In a composite-key table COMPLAINT_VOTE (TicketID, UserID) that also stores Title, Category, StudentName, and BlockName, a new student or a new complaint cannot be inserted until at least one vote is cast.
- *Update Anomaly:* Updating the Title or Category of Ticket 1003 requires updating all 30 vote rows associated with Ticket 1003. If even one row is missed, inconsistent titles exist for the same TicketID.
- *Delete Anomaly:* If all votes on a ticket are removed, deleting those rows also deletes the complaint title and student details.

Table Violating 2NF (Partial Dependencies on Composite Key (TicketID, UserID)):

TicketID	UserID	Title	Category	StudentName	BlockName	VoteType	VotedAt
1003	USR_257126003	Mess dinner paneer gravy sour	Mess	Aashi Agrawal	Charlie	Upvote	2026-09-26 21:15:00
1003	USR_257126004	Mess dinner paneer gravy sour	Mess	Aastha Bhattacharya	Charlie	Upvote	2026-09-26 21:20:00
1003	USR_257122235	Mess dinner paneer gravy sour	Mess	Raghav Sarda	Alpha	Upvote	2026-09-27 12:20:00

6.3 3NF Anomalies

- *Insert Anomaly:* If technician details (TechnicianID, TechnicianName, Trade, SupervisorWardenID) are stored directly inside the COMPLAINT table, a newly hired technician cannot be added to the database until they are assigned to an active complaint ticket.
- *Update Anomaly:* If technician Tukaram Gawade's supervising warden or trade changes, every historical complaint row assigned to him must be updated.
- *Delete Anomaly:* Deleting a resolved complaint ticket removes the technician's trade and supervisor information if that was the only ticket assigned to them.

Table Violating 3NF (Transitive Dependency TicketID -> TechnicianID -> Trade, SupervisorWardenID):

TicketID	Title	TechnicianID	TechnicianName	Trade	SupervisorWardenID
1001	Fortinet captive portal timeout	USR_STF111	Pravin Jagtap	Network Engineer	USR_STF101

1005	Lift 1 erratic jerking	USR_STF106	Tukaram Gawade	Electrician	USR_STF102
1006	Charlie-Study AC rattling noise	USR_STF107	Manoj Jadhav	Electrician	USR_STF104
1007	Beta-406 flush valve leakage	USR_STF108	Sandeep Patil	Plumber	USR_STF102

---

7. Functional Dependencies

Functional dependency refers to a relationship between attributes of a table where one attribute or set of attributes uniquely determines another.

Functional Dependencies in InHabit Schema:

1. **USER Table:**
   - UserID -> FirstName, MiddleName, LastName, InstitutionalEmail, PasswordHash, Role
   - InstitutionalEmail -> UserID, FirstName, MiddleName, LastName, PasswordHash, Role
2. **USER_PHONE Table:**
   - PhoneID -> UserID, PhoneNumber
   - (UserID, PhoneNumber) -> PhoneID
3. **STUDENT Table:**
   - UserID -> PRN, BlockName, FloorNumber, Wing
   - PRN -> UserID, BlockName, FloorNumber, Wing
4. **STAFF Table:**
   - UserID -> StaffID, Department
   - StaffID -> UserID, Department
5. **ADMIN Table:**
   - UserID -> AccessLevel
6. **WARDEN Table:**
   - UserID -> BlockInCharge

7. **TECHNICIAN Table:**
    - UserID -> Trade, SupervisorID
8. **COMPLAINT Table:**
    - TicketID -> Title, Description, Category, Status, CreatedAt, WardenConfirmed, StudentConfirmed, FiledBy
9. **ESCALATED_COMPLAINT Table:**
    - TicketID -> EscalatedAt
10. **SENSITIVE_COMPLAINT Table:**
    - TicketID -> IsAnonymous
11. **ATTACHMENT Table:**
    - AttachmentID -> TicketID, FileURL, FileType, FileSize_MB, UploadedAt
12. **VOTE Table:**
    - VoteID -> TicketID, UserID, VoteType, VotedAt
    - (TicketID, UserID) -> VoteID, VoteType, VotedAt
13. **STATUS_AUDIT_LOG Table:**
    - LogID -> TicketID, ChangedBy, OldStatus, NewStatus, ChangedAt
14. **TICKET_ASSIGNMENT Table:**
    - AssignmentID -> TicketID, WardenID, TechnicianID, AssignedAt, ResolutionNotes
15. **LOST_AND_FOUND Table:**
    - ItemID -> ItemName, Description, LocationFound, ItemType, Status, ReportedAt, ReportedBy

---

8. Database Normalization

Normalization splits and structures tables to remove redundancy and prevent data anomalies.

8.1 First Normal Form (1NF)

- *Rule:* A relation is in 1NF if it contains only atomic (single-valued) attributes and no repeating groups.
- *Application in InHabit:* Multi-valued attributes PhoneNumber and EvidenceFiles are decomposed into separate tables USER_PHONE and ATTACHMENT, and composite attributes (UserName, HostelLocation) are split into atomic columns.

Decomposed Atomic Tables (1NF):

USER_PHONE Table:

PhoneID	UserID	PhoneNumber
1	USR_STF101	9876000101
462	USR_257122235	9876122235
466	USR_257122247	9876122247

ATTACHMENT Table:

AttachmentID	TicketID	FileURL	FileType	FileSize_MB	UploadedAt
ATT101	1001	https://inhabit.campus/storage/att_1001_wifi_speedtest.png	image/png	2.40	2026-09-24 16:42:00
ATT102	1003	https://inhabit.campus/storage/att_1003_turbid_water.jpg	image/jpeg	3.80	2026-09-26 21:12:00

8.2 Second Normal Form (2NF)

- *Rule:* A relation is in 2NF if it is in 1NF and contains no partial dependencies (no non-prime attribute depends on a proper subset of a composite candidate key).
- *Application in InHabit:* In the voting relation with candidate key (TicketID, UserID), complaint details (Title, Category) depend only on TicketID, and student details (PRN, BlockName) depend only on UserID. Separating COMPLAINT, STUDENT, and VOTE eliminates partial dependencies.

Decomposed Tables (2NF):

VOTE Table:

VoteID	TicketID	UserID	VoteType	VotedAt
VOT101	1003	USR_257126003	Upvote	2026-09-26 21:15:00
VOT128	1003	USR_257122235	Upvote	2026-09-27 12:20:00
VOT129	1003	USR_257122247	Upvote	2026-09-27 12:30:00

8.3 Third Normal Form (3NF)

- *Rule:* A relation is in 3NF if it is in 2NF and contains no transitive dependencies (for every non-trivial FD X -> Y, X is a superkey or Y is a prime attribute).
- *Application in InHabit:* Technician attributes (Trade, SupervisorID) transitively dependent on TicketID via TechnicianID are decomposed into TECHNICIAN and TICKET_ASSIGNMENT.

Decomposed Tables (3NF):

TECHNICIAN Table:

UserID	Trade	SupervisorID
USR_STF106	Electrician	USR_STF102
USR_STF108	Plumber	USR_STF102
USR_STF111	Network Engineer	USR_STF101

TICKET_ASSIGNMENT Table:

AssignmentID	TicketID	WardenID	TechnicianID	AssignedAt	ResolutionNotes
ASN101	1001	USR_STF101	USR_STF111	2026-09-24 17:30:00	Configured DHCP lease time and restarted floor access point
ASN102	1005	USR_STF102	USR_STF106	2026-09-27 19:00:00	Elevator motor brake pads inspected; guide rails lubricated

8.4 Boyce-Codd Normal Form (BCNF) Verification

- *Rule:* For every non-trivial functional dependency X -> Y, X must be a superkey.
- *Verification:* Across all 15 normalized tables in InHabitDBNew, every determinant X (UserID, InstitutionalEmail, PRN, StaffID, TicketID, VoteID, AttachmentID, LogID, AssignmentID, ItemID, PhoneID) is a candidate key. Thus, the entire database schema is in **BCNF**.

---

9. Application of Codd's Rules

I. Information Rule:
Data stored in a Relational model must be a value of some cell of a table.

- *Followed:* All user profiles, room allocations, complaints, votes, and audit logs are stored as atomic values in relational tables in MySQL.

II. Guaranteed Access Rule:
Every data element must be accessible by table name, its primary key, and name of attribute whose value is to be determined.

- *Followed:* Every scalar value is uniquely addressable via table name, primary key (UserID, TicketID, etc.), and column name.

III. Systematic Treatment of NULL values:
NULL values in the database only correspond to missing, unknown, or not applicable values.

- *Followed:* NULL is used systematically for optional middle names (MiddleName) and top-level supervisor references.

IV. Active Online Catalog:
The structure of the database must be stored in an online catalog which can be queried by authorized users.

- *Followed:* MySQL maintains INFORMATION_SCHEMA tables that can be queried by authorized users.

V. Comprehensive Data Sub-Language Rule:

A database can only be accessed using a language having a linear syntax that supports data definition, data manipulation, and transaction management operations.

- *Followed:* MySQL implements SQL supporting DDL, DML, DQL, and transactional operations.

VI. View Updating Rule:

Different views created for various purposes should be automatically updatable by the system.

- *Not Followed:* While simple views are updatable, joined views like AssignedTicketsView are not directly updatable in MySQL.

VII. High-Level Insert, Update, and Delete Rule:

The Relational Model should support insert, delete, update operations at each level of relations.

- *Followed:* Set-level batch inserts (e.g., 500 student records), updates, deletes, and UNION operations are supported in MySQL.

VIII. Physical Data Independence:

Any modification in the physical location of a table should not enforce modification at application level.

- Followed: Storage engines and B-tree index files are managed internally by MySQL without impacting Java JDBC queries.

IX. Logical Data Independence:

Any modification in the logical or conceptual schema of a table should not enforce modification at application level.

- *Not Followed:* Splitting or merging base tables requires corresponding updates in DAO SQL queries.

X. Integrity Independence:

Integrity constraints must be specified separately from application programs and stored in the catalog.

- *Not Fully Followed:* Primary keys, foreign keys (ON DELETE CASCADE), and UNIQUE(TicketID, UserID) are enforced in the database schema, though some UI validation also exists in the Java client.

XI. Distribution Independence:
Distribution of data over various locations is not visible to end-users.

- *Followed:* JDBC connection URLs (db.properties) abstract whether the MySQL server runs locally or remotely.

XII. Non-Subversion Rule:
If a system has an interface that provides access to low-level records, then the interface must not be able to subvert the system and bypass security and integrity constraints.

- *Followed:* All data access passes through SQL and JDBC PreparedStatement queries, preserving all foreign key and trigger constraints.

---

10. Implementation of Tables in MySQL

10.1 MySQL DDL Implementation (InHabitDBNew.sql):

```
create database InHabitDBNew;
use InHabitDBNew;

-- Table 01: USER
CREATE TABLE USER(
 USERID VARCHAR(30) PRIMARY KEY,
 FIRSTNAME VARCHAR(50) not null,
 MIDDLENAME VARCHAR(50) null,
 LASTNAME VARCHAR(50) null,
 INSTITUTIONALEMAIL VARCHAR(100) unique not null,
 PASSWORDHASH VARCHAR(100) not null,
 ROLE VARCHAR(30) not null
);

-- Table 02: USER_PHONE
CREATE TABLE USER_PHONE(
 PHONEID INT PRIMARY KEY,
 USERID VARCHAR(30), FOREIGN KEY(USERID) references USER(USERID) on delete
cascade,
 PHONENUMBER VARCHAR(20) not null
);
```

```
-- Table 03: STUDENT
CREATE TABLE STUDENT(
 USERID VARCHAR(30) PRIMARY KEY,
 PRN VARCHAR(20) unique not null,
 BLOCKNAME VARCHAR(30) not null,
 FLOORNUMBER INT not null,
 WING VARCHAR(20) not null,
 FOREIGN KEY(USERID) references USER(USERID) on delete cascade
);

-- Table 04: STAFF
CREATE TABLE STAFF(
 USERID VARCHAR(30) PRIMARY KEY,
 STAFFID VARCHAR(20) unique not null,
 DEPARTMENT VARCHAR(50) not null,
 FOREIGN KEY(USERID) references USER(USERID) on delete cascade
);

-- Table 05: ADMIN
CREATE TABLE ADMIN(
 USERID VARCHAR(30) PRIMARY KEY,
 ACCESSLEVEL VARCHAR(30) not null,
 FOREIGN KEY(USERID) references STAFF(USERID) on delete cascade
);

-- Table 06: WARDEN
CREATE TABLE WARDEN(
 USERID VARCHAR(30) PRIMARY KEY,
 BLOCKINCHARGE VARCHAR(50) not null,
 FOREIGN KEY(USERID) references STAFF(USERID) on delete cascade
);

-- Table 07: TECHNICIAN
CREATE TABLE TECHNICIAN(
 USERID VARCHAR(30) PRIMARY KEY,
 TRADE VARCHAR(50) not null,
 SUPERVISORID VARCHAR(30) null,
 FOREIGN KEY(USERID) references STAFF(USERID) on delete cascade,
 FOREIGN KEY(SUPERVISORID) references WARDEN(USERID) on delete set null
);

-- Table 08: COMPLAINT
CREATE TABLE COMPLAINT(
 TICKETID INT PRIMARY KEY,
 TITLE VARCHAR(150) not null,
 DESCRIPTION TEXT not null,
 CATEGORY VARCHAR(50) not null,
 STATUS VARCHAR(30) default 'Open',
```

```
 CREATEDAT DATETIME not null,
 WARDENCONFIRMED BOOLEAN default false,
 STUDENTCONFIRMED BOOLEAN default false,
 FILEDBY VARCHAR(30), FOREIGN KEY(FILEDBY) references STUDENT(USERID) on delete
cascade
);

-- Table 09: ESCALATED_COMPLAINT
CREATE TABLE ESCALATED_COMPLAINT(
 TICKETID INT PRIMARY KEY,
 ESCALATEDAT DATETIME not null,
 FOREIGN KEY(TICKETID) references COMPLAINT(TICKETID) on delete cascade
);

-- Table 10: SENSITIVE_COMPLAINT
CREATE TABLE SENSITIVE_COMPLAINT(
 TICKETID INT PRIMARY KEY,
 ISANONYMOUS BOOLEAN default true,
 FOREIGN KEY(TICKETID) references COMPLAINT(TICKETID) on delete cascade
);

-- Table 11: ATTACHMENT
CREATE TABLE ATTACHMENT(
 ATTACHMENTID VARCHAR(20) PRIMARY KEY,
 TICKETID INT, FOREIGN KEY(TICKETID) references COMPLAINT(TICKETID) on delete
cascade,
 FILEURL VARCHAR(255) not null,
 FILETYPE VARCHAR(50) not null,
 FILESIZE_MB DECIMAL(5,2) not null,
 UPLOADEDAT DATETIME not null
);

-- Table 12: VOTE
CREATE TABLE VOTE(
 VOTEID VARCHAR(20) PRIMARY KEY,
 TICKETID INT, FOREIGN KEY(TICKETID) references COMPLAINT(TICKETID) on delete
cascade,
 USERID VARCHAR(30), FOREIGN KEY(USERID) references STUDENT(USERID) on delete
cascade,
 VOTETYPE VARCHAR(20) default 'Upvote',
 VOTEDAT DATETIME not null,
 UNIQUE(TICKETID, USERID)
);

-- Table 13: STATUS_AUDIT_LOG
CREATE TABLE STATUS_AUDIT_LOG(
 LOGID INT PRIMARY KEY,
 TICKETID INT, FOREIGN KEY(TICKETID) references COMPLAINT(TICKETID) on delete
```

```
cascade,
 CHANGEDBY VARCHAR(30) null, FOREIGN KEY(CHANGEDBY) references USER(USERID) on
delete set null,
 OLDSTATUS VARCHAR(30) not null,
 NEWSTATUS VARCHAR(30) not null,
 CHANGEDAT DATETIME not null
);

-- Table 14: TICKET_ASSIGNMENT
CREATE TABLE TICKET_ASSIGNMENT(
 ASSIGNMENTID VARCHAR(20) PRIMARY KEY,
 TICKETID INT, FOREIGN KEY(TICKETID) references COMPLAINT(TICKETID) on delete
cascade,
 WARDENID VARCHAR(30) null, FOREIGN KEY(WARDENID) references WARDEN(USERID) on
delete set null,
 TECHNICIANID VARCHAR(30) null, FOREIGN KEY(TECHNICIANID) references
TECHNICIAN(USERID) on delete set null,
 ASSIGNEDAT DATETIME not null,
 RESOLUTIONNOTES TEXT null
);

-- Table 15: LOST_AND_FOUND
CREATE TABLE LOST_AND_FOUND(
 ITEMID VARCHAR(20) PRIMARY KEY,
 ITEMNAME VARCHAR(100) not null,
 DESCRIPTION TEXT not null,
 LOCATIONFOUND VARCHAR(100) not null,
 ITEMTYPE VARCHAR(20) not null,
 STATUS VARCHAR(20) default 'Open',
 REPORTEDAT DATETIME not null,
 REPORTEDBY VARCHAR(30) null, FOREIGN KEY(REPORTEDBY) references USER(USERID)
on delete set null
);
```

10.2 Table Outputs (Sample Verification):

1. USER (Sample + Total Count)

```
select UserID, FirstName, LastName, InstitutionalEmail, Role
from user
limit 19;

select count(*) as TotalUsers
from user;
```

UserID	FirstName	LastName	InstitutionalEmail	Role
USR_257122001	A	Visvajit	a.visvajit.btech25@college.edu.in	Student
USR_257122002	Aarushi	Raizada	aarushi.raizada.btech25@college.edu.in	Student
USR_257122005	Aashray	Kayastha	aashray.kayastha.btech25@college.edu.in	Student
USR_257122006	Aatharv	Khandave	aatharv.khandave.btech25@college.edu.in	Student
USR_257122008	Abdulrahman	Awadhi	abdulrahman.awadhi.btech25@college.edu.in	Student
USR_257122009	Abel	Joseph	abel.joseph.btech25@college.edu.in	Student
USR_257122012	Abhishek	Singh	abhishek.singh.btech25@college.edu.in	Student
USR_257122013	Abhyuday	Tripathi	abhyuday.tripathi.btech25@college.edu.in	Student
USR_257122014	Abishek	Yadav	abishek.yadav.btech25@college.edu.in	Student
USR_257122015	Abobakr	Mahir	abobakr.mahir.btech25@college.edu.in	Student
USR_257122016	Adarsh	Bimal	adarsh.bimal.btech25@college.edu.in	Student
USR_257122019	Aditya	Barnwal	aditya.barnwal.btech25@college.edu.in	Student
USR_257122021	Aditya	Jaiswal	aditya.jaiswal.btech25@college.edu.in	Student
USR_257122022	Aditya	Mandal	aditya.mandal.btech25@college.edu.in	Student
USR_257122023	Advait	Dhanawade	advait.dhanawade.btech25@college.edu.in	Student
USR_257122024	Advaith	Dasgupta	advaith.dasgupta.btech25@college.edu.in	Student
USR_257122025	Agam	Oze	agam.oze.btech25@college.edu.in	Student
USR_257122026	Ahasas	Singh	ahasas.singh.btech25@college.edu.in	Student
USR_257122028	Akmal	Abbas	akmal.abbas.btech25@college.edu.in	Student

TotalUsers
520

2. STUDENT (Sample + Total Count)

```
select UserID, PRN, BlockName, FloorNumber, Wing
from student
limit 19;

select count(*) as TotalStudents
from student;
```

UserID	PRN	BlockName	FloorNumber	Wing
USR_257122001	257122001	Alpha	5	508
USR_257122002	257122002	Charlie	4	408
USR_257122005	257122005	Alpha	5	508
USR_257122006	257122006	Alpha	5	508
USR_257122008	257122008	Alpha	5	508
USR_257122009	257122009	Alpha	5	509
USR_257122012	257122012	Alpha	5	509
USR_257122013	257122013	Alpha	5	509
USR_257122014	257122014	Alpha	5	509
USR_257122015	257122015	Alpha	5	510
USR_257122016	257122016	Alpha	5	510
USR_257122019	257122019	Alpha	5	510
USR_257122021	257122021	Alpha	5	510
USR_257122022	257122022	Beta	1	101
USR_257122023	257122023	Beta	1	101
USR_257122024	257122024	Beta	1	101
USR_257122025	257122025	Beta	1	101
USR_257122026	257122026	Beta	1	102
USR_257122028	257122028	Beta	1	102

TotalStudents
500

3. COMPLAINT (All 15 Records)

```
select TicketID, Title, Category, Status, CreatedAt, FiledBy
from complaint;
```

TicketID	Title	Category	Status	CreatedAt	FiledBy
1001	Fortinet captive portal timeout on 3rd floor Wi-Fi	Internet	In Progress	2026-09-24 16:40:00	USR_257122235
1002	Solar hot water pipeline cold in washroom	Plumbing	Under Review	2026-09-25 07:15:00	USR_257122247
1003	Mess dinner paneer gravy sour and undercooked rotis	Mess	Open	2026-09-26 21:10:00	USR_257122001
1004	Balcony sliding window latch broken	Cleanliness	Open	2026-09-27 11:30:00	USR_257122006
1005	Lift 1 erratic jerking between 2nd and 3rd floor	Electrical	In Progress	2026-09-27 18:25:00	USR_257122013
1006	Charlie-Study AC excessive rattling noise	Electrical	Resolved	2026-09-28 14:10:00	USR_257126003
1007	Beta-406 flush valve leakage	Plumbing	In Progress	2026-09-28 16:30:00	USR_257126004
1008	Delta-401 water cooler filter smell	Cleanliness	Open	2026-09-29 08:45:00	USR_257126008
1009	Alpha-409 blinking tube light fixture	Electrical	Resolved	2026-09-29 19:20:00	USR_257126010
1010	Gate 1 biometric scanner rebooting	Security	In Progress	2026-09-30 20:15:00	USR_257122002
1011	Charlie wing 2nd floor LAN port disconnected	Internet	In Progress	2026-10-01 09:00:00	USR_257122019
1012	Central mess breakfast milk dispenser clogged	Mess	Resolved	2026-10-01 08:15:00	USR_257122025
1013	Beta wing washbasin tap leaking heavily	Plumbing	Open	2026-10-01 11:20:00	USR_257122030
1014	Delta corridor stairwell emergency light offline	Electrical	Open	2026-10-01 14:50:00	USR_257122045
1015	Charlie wing ground floor exit door magnetic latch loose	Security	Resolved	2026-10-01 22:30:00	USR_257122055

11. Queries, Functions, Procedures & Triggers Implemented

11.1 Analytical Queries Implemented:

Query 1: Which students reside in Alpha Block, Room 101?

```
select user.FirstName, user.LastName, student.PRN, student.BlockName,
student.FloorNumber, student.Wing
from user
inner join student on user.UserID = student.UserID
where student.BlockName = "Alpha"
and student.Wing = "101";
```

FirstName	LastName	PRN	BlockName	FloorNumber	Wing
Yatharth	Shah	257122223	Alpha	1	101
Raghav	Sarda	257122235	Alpha	1	101
Parth	Kulkarni	257122299	Alpha	1	101

Query 2: Find all complaints which are currently 'Open' or 'In Progress'

```
select *
from complaint
where Status = "Open" or Status = "In Progress";
```

TICKETID	TITLE	DESCRIPTION	CATEGORY	STATUS	CREATEDAT	WARDENCONFIRMED	STUDENTCONFIRMED	FILEDBY
1001	Fortinet captive portal timeout on 3rd floor Wi-Fi	Cyberoam/Fortinet login gateway times out continuously on 3rd floor access points during peak hours.	Internet	In Progress	2026-09-24 16:40:00	1	0	USR_257122235
1003	Mess dinner paneer gravy sour and undercooked rotis	Wednesday evening dinner paneer tasted sour and rotis were undercooked; multiple students reported stomach issues.	Mess	Open	2026-09-26 21:10:00	0	1	USR_257122001
1004	Balcony sliding window latch broken	Sliding window latch in room 108 is damaged; window does not lock securely against monsoon rain and wind.	Cleanliness	Open	2026-09-27 11:30:00	0	0	USR_257122006
1005	Lift 1 erratic jerking between 2nd and 3rd floor	Passenger elevator shakes abruptly between 2nd and 3rd floors before leveling at the landing.	Electrical	In Progress	2026-09-27 18:25:00	0	0	USR_257122013
1007	Beta-406 flush valve leakage	Dual flush valve continuously trickles water into toilet bowl, causing wastage and damp flooring.	Plumbing	In Progress	2026-09-28 16:30:00	0	0	USR_257126004
1008	Delta-401 water cooler filter smell	Drinking water dispenser on 4th floor corridor has unpleasant odor and stale taste; filter replacement needed.	Cleanliness	Open	2026-09-29 08:45:00	0	0	USR_257126008
1010	Gate 1 biometric scanner rebooting	Main entrance biometric attendance unit repeatedly restarts on user thumbprint verification during evening rush.	Security	In Progress	2026-09-30 20:15:00	0	0	USR_257122002
1011	Charlie wing 2nd floor LAN port disconnected	Wall Ethernet jack RJ45 connector damaged and does not establish connection with hostel subnet.	Internet	In Progress	2026-10-01 09:00:00	0	0	USR_257122019
1013	Beta wing washbasin tap leaking heavily	Pillar tap in 3rd floor washroom cannot be shut tightly and leaks a steady stream into the sink.	Plumbing	Open	2026-10-01 11:20:00	0	0	USR_257122030
1014	Delta corridor stairwell emergency light offline	Battery backup emergency LED panel on west stairwell fails to illuminate during scheduled power cut.	Electrical	Open	2026-10-01 14:50:00	0	0	USR_257122045

Query 3: Display complaints along with the student name who filed them

```
select complaint.TicketID, complaint.Title, complaint.Category,
complaint.Status, user.FirstName, user.LastName
from complaint
inner join student on complaint.FiledBy = student.UserID
inner join user on student.UserID = user.UserID;
```

TicketID	Title	Category	Status	FirstName	LastName
1001	Fortinet captive portal timeout on 3rd floor Wi-Fi	Internet	In Progress	Raghav	Sarda
1002	Solar hot water pipeline cold in washroom	Plumbing	Under Review	Aniruddha	Singh
1003	Mess dinner paneer gravy sour and undercooked rotis	Mess	Open	A	Visvajit
1004	Balcony sliding window latch broken	Cleanliness	Open	Aatharv	Khandave
1005	Lift 1 erratic jerking between 2nd and 3rd floor	Electrical	In Progress	Abhyuday	Tripathi
1006	Charlie-Study AC excessive rattling noise	Electrical	Resolved	Aashi	Agrawal
1007	Beta-406 flush valve leakage	Plumbing	In Progress	Aastha	Bhattacharya
1008	Delta-401 water cooler filter smell	Cleanliness	Open	Abhinav	Mankar
1009	Alpha-409 blinking tube light fixture	Electrical	Resolved	Adeeb	Mulla
1010	Gate 1 biometric scanner rebooting	Security	In Progress	Aarushi	Raizada
1011	Charlie wing 2nd floor LAN port disconnected	Internet	In Progress	Aditya	Barnwal
1012	Central mess breakfast milk dispenser clogged	Mess	Resolved	Agam	Oze
1013	Beta wing washbasin tap leaking heavily	Plumbing	Open	Akul	Agarwal
1014	Delta corridor stairwell emergency light offline	Electrical	Open	Arhaan	Khan
1015	Charlie wing ground floor exit door magnetic latch loose	Security	Resolved	Atharva	Ambilwade

---

Query 4: Group By, Having & Order By: Give category-wise complaint count

```
select Category, count(TicketID) as TotalComplaints
from complaint
group by Category
order by TotalComplaints desc;
```

Category	TotalComplaints
Electrical	4
Plumbing	3
Internet	2
Mess	2
Cleanliness	2
Security	2

Query 5: Find tickets that have received more than 20 votes

```
select TicketID, count(VoteID) as TotalVotes
from vote
group by TicketID
having count(VoteID) > 20;
```

TicketID	TotalVotes
1003	30

11.2 View Implementation: Create a view to display all assigned tickets with Warden and Technician details

```
create or replace view AssignedTicketsView as

select complaint.TicketID, complaint.Title, complaint.Category,
complaint.Status, ticket_assignment.WardenID, ticket_assignment.TechnicianID,
ticket_assignment.ResolutionNotes

from complaint

inner join ticket_assignment on complaint.TicketID =
ticket_assignment.TicketID;
```

TicketID	Title	Category	Status	WardenID	TechnicianID	ResolutionNotes
1001	Fortinet captive portal timeout on 3rd floor Wi-Fi	Internet	In Progress	USR_STF101	USR_STF111	Configured DHCP lease time and restarted floor access point
1005	Lift 1 erratic jerking between 2nd and 3rd floor	Electrical	In Progress	USR_STF102	USR_STF106	Elevator motor brake pads inspected; guide rails lubricated
1006	Charlie-Study AC excessive rattling noise	Electrical	Resolved	USR_STF104	USR_STF107	AC compressor mount tightened and condenser fan re-aligned
1007	Beta-406 flush valve leakage	Plumbing	In Progress	USR_STF102	USR_STF108	Replaced damaged internal rubber washer in dual-flush cistern
1009	Alpha-409 blinking tube light fixture	Electrical	Resolved	USR_STF103	USR_STF107	Replaced flickering starter and ballast with 20W LED tube
1010	Gate 1 biometric scanner rebooting	Security	In Progress	USR_STF101	USR_STF117	Replaced faulty 12V 2A power adapter for biometric scanner
1011	Charlie wing 2nd floor LAN port disconnected	Internet	In Progress	USR_STF104	USR_STF112	Re-crimped RJ45 keystones and patched line to switch port
1012	Central mess breakfast milk dispenser clogged	Mess	Resolved	USR_STF101	USR_STF113	Cleared blockage in urn dispenser faucet and steam-sterilized
1015	Charlie wing ground floor exit door magnetic latch loose	Security	Resolved	USR_STF105	USR_STF120	Realigned strike armature plate and secured hex mounting screws

---

11.3 Function Implementation: Calculate Days Pending for a given TicketID

```
DELIMITER //
CREATE FUNCTION fn_DaysPending(p_ticket_id INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE v_days INT;
    SELECT DATEDIFF(CURRENT_DATE(), DATE(CreatedAt)) INTO v_days
    FROM COMPLAINT
    WHERE TicketID = p_ticket_id;
    RETURN IFNULL(v_days, 0);
END //
DELIMITER ;

select TicketID, Title, Status, fn_DaysPending(TicketID) as DaysPending from
complaint;
```

TicketID	Title	Status	DaysPending
1001	Fortinet captive portal timeout on 3rd floor Wi-Fi	In Progress	25
1002	Solar hot water pipeline cold in washroom	Under Review	24
1003	Mess dinner paneer gravy sour and undercooked rotis	Open	23
1004	Balcony sliding window latch broken	Open	22
1005	Lift 1 erratic jerking between 2nd and 3rd floor	In Progress	22
1006	Charlie-Study AC excessive rattling noise	Resolved	21
1007	Beta-406 flush valve leakage	In Progress	21
1008	Delta-401 water cooler filter smell	Open	20
1009	Alpha-409 blinking tube light fixture	Resolved	20
1010	Gate 1 biometric scanner rebooting	In Progress	19
1011	Charlie wing 2nd floor LAN port disconnected	In Progress	18
1012	Central mess breakfast milk dispenser clogged	Resolved	18
1013	Beta wing washbasin tap leaking heavily	Open	18
1014	Delta corridor stairwell emergency light offline	Open	18
1015	Charlie wing ground floor exit door magnetic latch loose	Resolved	18

11.4 Stored Procedure Implementation: Assign a Technician to a Complaint and log the status transition

```
DELIMITER //
CREATE PROCEDURE sp_AssignTicket(
    IN p_assignment_id VARCHAR(20),
    IN p_ticket_id INT,
    IN p_warden_id VARCHAR(30),
    IN p_tech_id VARCHAR(30),
    IN p_notes TEXT
)
BEGIN
    DECLARE v_old_status VARCHAR(30);
    DECLARE v_next_log_id INT;

    SELECT Status INTO v_old_status FROM COMPLAINT WHERE TicketID =
p_ticket_id;
    SELECT IFNULL(MAX(LogID), 0) + 1 INTO v_next_log_id FROM STATUS_AUDIT_LOG;

    INSERT INTO TICKET_ASSIGNMENT (AssignmentID, TicketID, WardenID,
TechnicianID, AssignedAt, ResolutionNotes)
    VALUES (p_assignment_id, p_ticket_id, p_warden_id, p_tech_id, NOW(),
p_notes);

    UPDATE COMPLAINT SET Status = 'In Progress' WHERE TicketID = p_ticket_id;

    INSERT INTO STATUS_AUDIT_LOG (LogID, TicketID, ChangedBy, OldStatus,
```

```
NewStatus, ChangedAt)
    VALUES (v_next_log_id, p_ticket_id, p_warden_id, v_old_status, 'In
Progress', NOW());
END //
DELIMITER ;

CALL sp_AssignTicket('ASN110', 1004, 'USR_STF102', 'USR_STF108', 'Inspecting
damaged window latch and alignment');
select * from ticket_assignment where AssignmentID = "ASN110";
select TicketID, Status from complaint where TicketID = 1004;
select * from status_audit_log where TicketID = 1004 order by LogID desc limit
1;
```

```
+------------+----------+-----------+-------------+---------------------+-----------------------------------------+
| ASSIGNMENTID | TICKETID | WARDENID   | TECHNICIANID | ASSIGNEDAT          | RESOLUTIONNOTES                         |
+------------+----------+-----------+-------------+---------------------+-----------------------------------------+
| ASN110       |     1004 | USR_STF102 | USR_STF108   | 2026-10-19 19:01:44 | Inspecting damaged window latch and alignment |
+------------+----------+-----------+-------------+---------------------+-----------------------------------------+
+----------+------------+
| TicketID | Status      |
+----------+------------+
|     1004 | In Progress |
+----------+------------+
```

11.5 Trigger Implementation: Automatically insert into ESCALATED_COMPLAINT when a ticket reaches 30 votes

```
DELIMITER //
CREATE TRIGGER trg_AutoEscalateComplaint
AFTER INSERT ON VOTE
FOR EACH ROW
BEGIN
    DECLARE v_vote_count INT;
    SELECT COUNT(*) INTO v_vote_count FROM VOTE WHERE TicketID = NEW.TicketID;

    IF v_vote_count >= 30 THEN
        INSERT IGNORE INTO ESCALATED_COMPLAINT (TicketID, EscalatedAt)
        VALUES (NEW.TicketID, NOW());
    END IF;
END //
DELIMITER ;

-- Check current votes for Ticket 1003 (already has 30 votes and is in
escalated_complaint)
select TicketID, count(VoteID) as TotalVotes
from vote
where TicketID = 1003;

-- Test on Ticket 1001 by inserting a vote to reach or check escalation
insert into vote (VoteID, TicketID, UserID, VoteType, VotedAt)
```

```
values ("VOT999", 1001, "USR_257126003", "Upvote", now());

-- Verify if escalated:
select * from escalated_complaint where TicketID = 1001;
```

```
+-------+----------+-----------+----------+------------+---------------------+
| LOGID | TICKETID | CHANGEDBY | OLDSTATUS | NEWSTATUS  | CHANGEDAT           |
+-------+----------+-----------+----------+------------+---------------------+
|    30 |     1004 | USR_STF102 | Open     | In Progress | 2026-10-19 19:01:44 |
+-------+----------+-----------+----------+------------+---------------------+
+----------+------------+
| TicketID | TotalVotes |
+----------+------------+
|     1003 |         30 |
+----------+------------+
```

---

12. Project Demonstration

The project includes a full Java Swing and JDBC application (InHabit.jar / src/inhabit/) connected to the InHabitDBNew MySQL database:

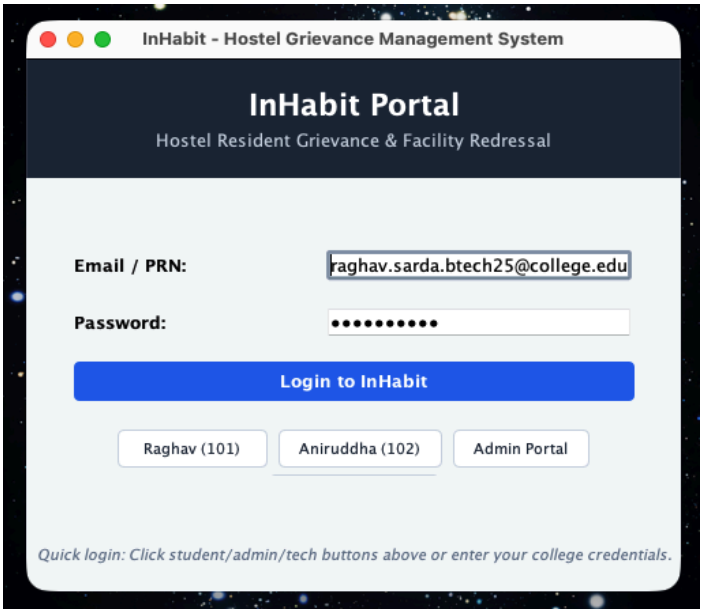
- **Database Layer (DBConnection.java, ComplaintDAO.java, UserDAO.java):** Executes parameterized SQL queries, joins, and transactions against the 15-table normalized schema.
- **Role-Based Dashboards (LoginFrame.java, StudentDashboard.java, AdminDashboard.java, TechnicianDashboard.java):** Provides dedicated interfaces for students (filing tickets, upvoting, Lost & Found), wardens/admins (assigning technicians, reviewing escalated tickets), and technicians (updating resolution notes).
- **Automated Escalation Daemon (EscalationThread.java) & Trigger (trg_AutoEscalateComplaint):** Demonstrates real-time auto-escalation when a complaint reaches >= 30 genuine votes.
- **Verification Suite (SmokeTest.java & InHabitDemo.java):** Validates end-to-end connectivity, authentication, voting constraints, and audit trail logging.

GitHub Link: https://github.com/iamthersforever/InHabit

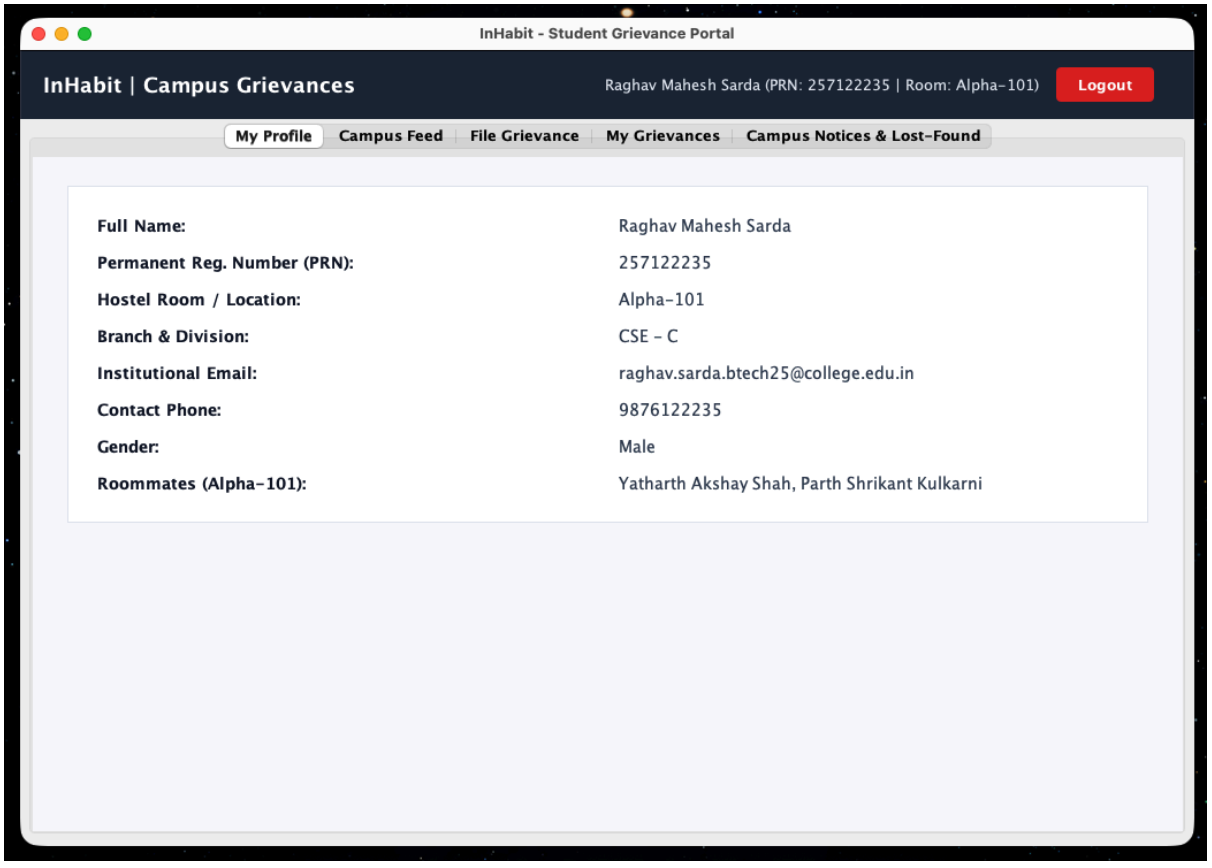
Screenshots:

Student:

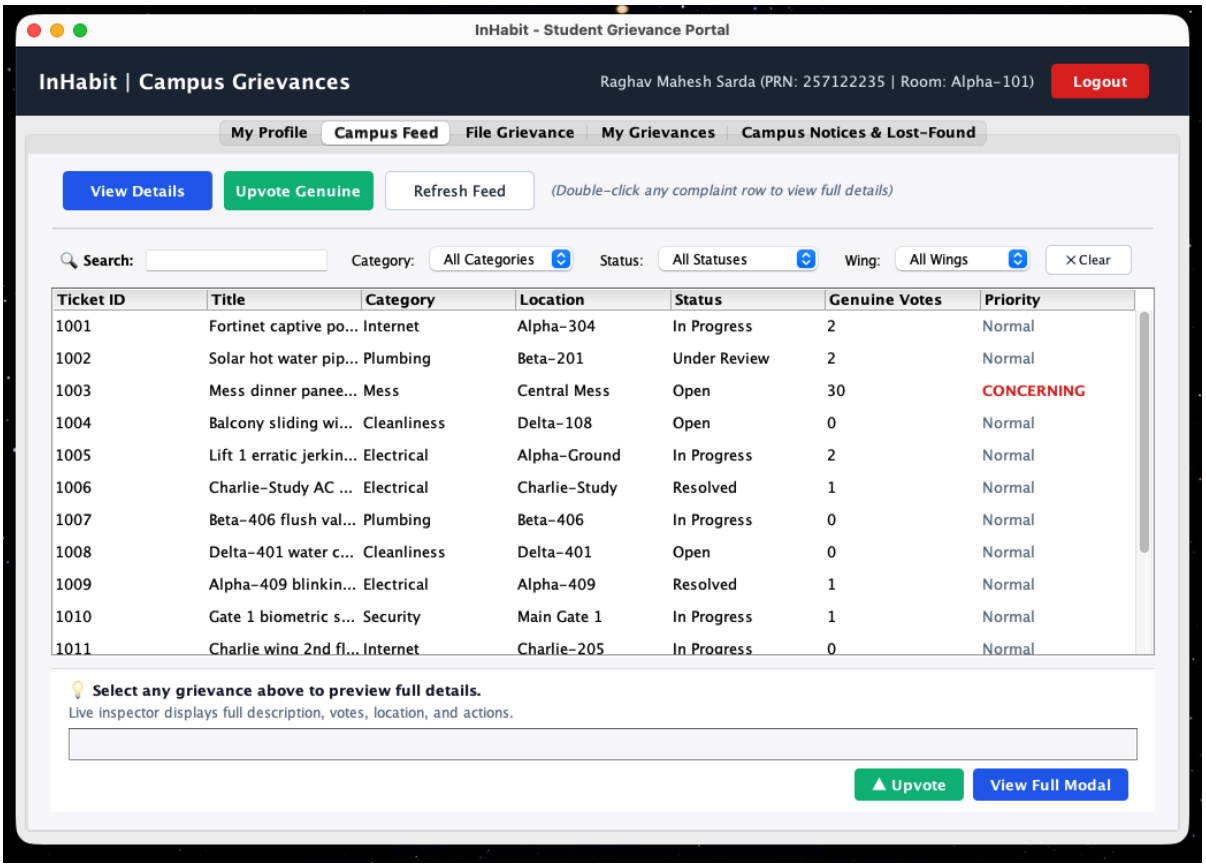
1. Login Page:

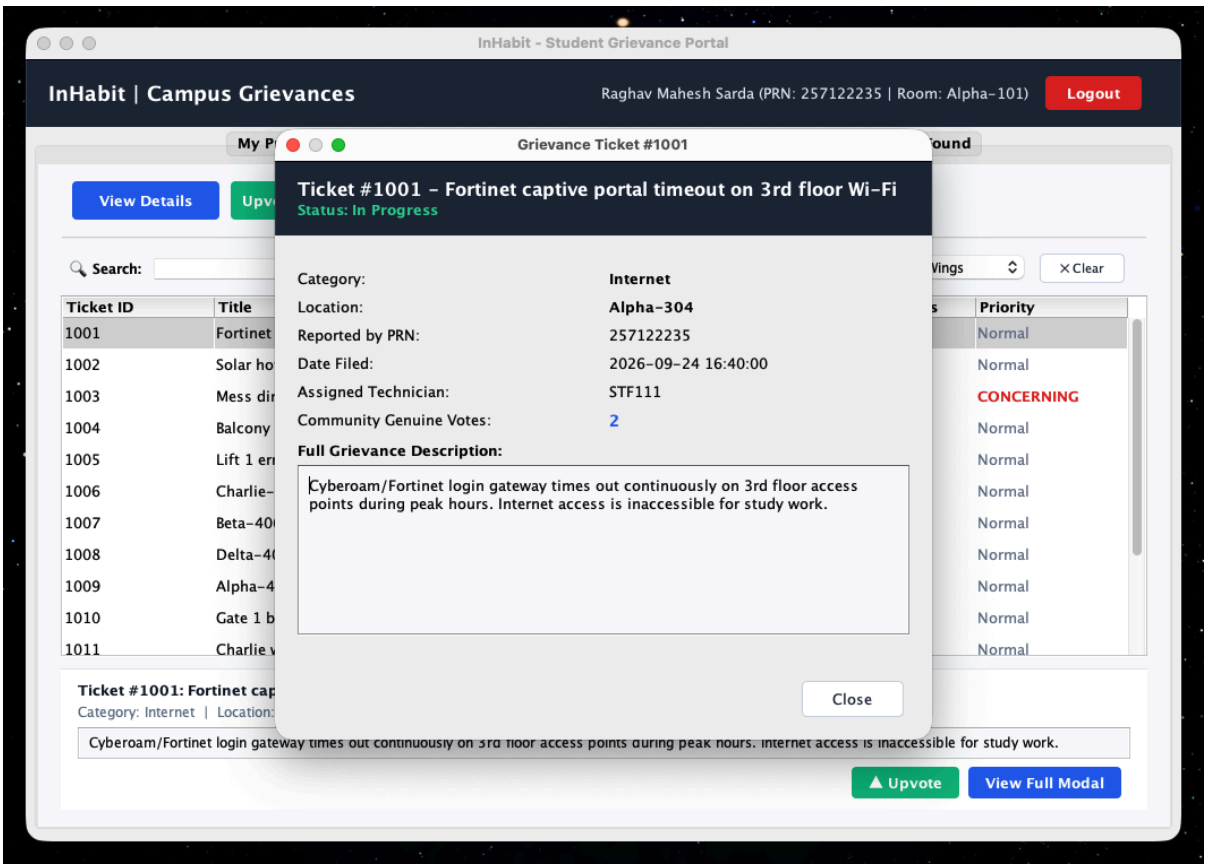


2. My Profile:

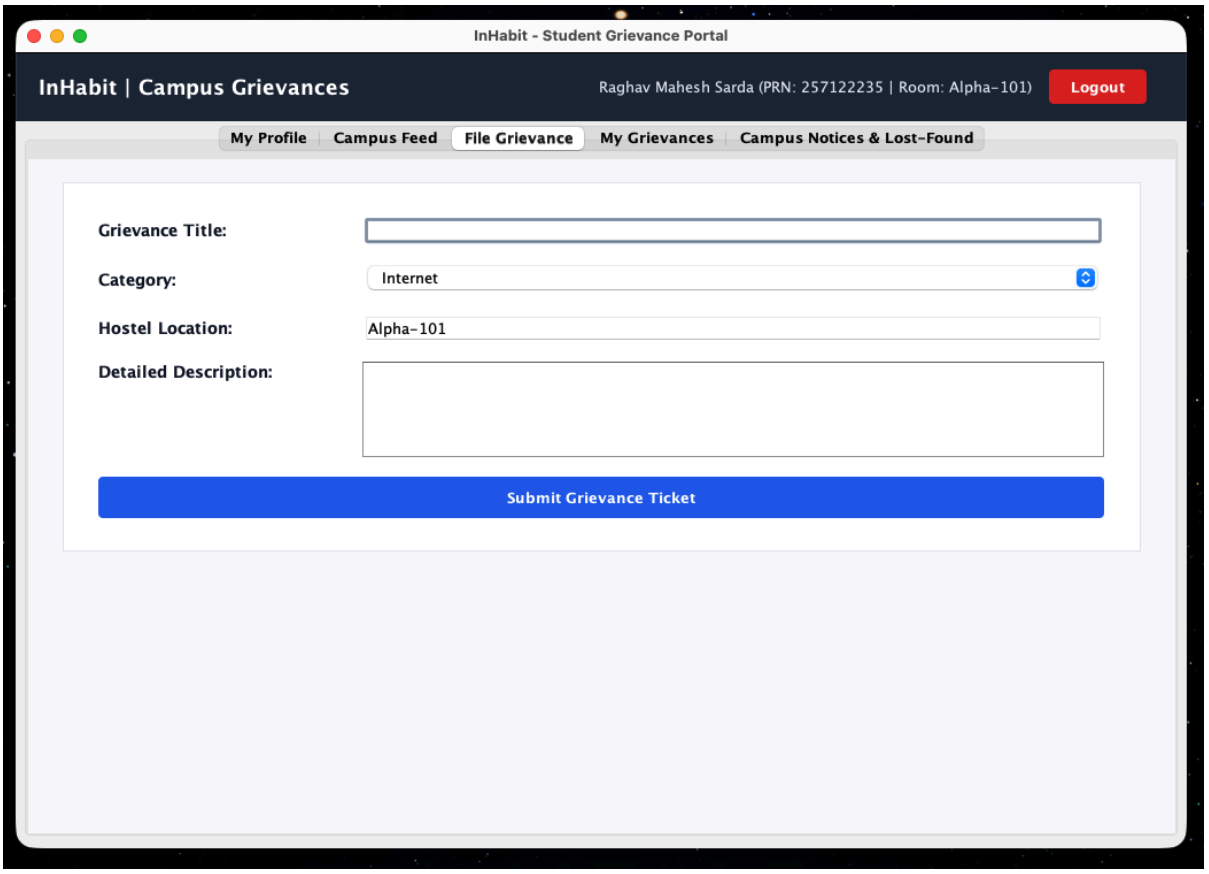


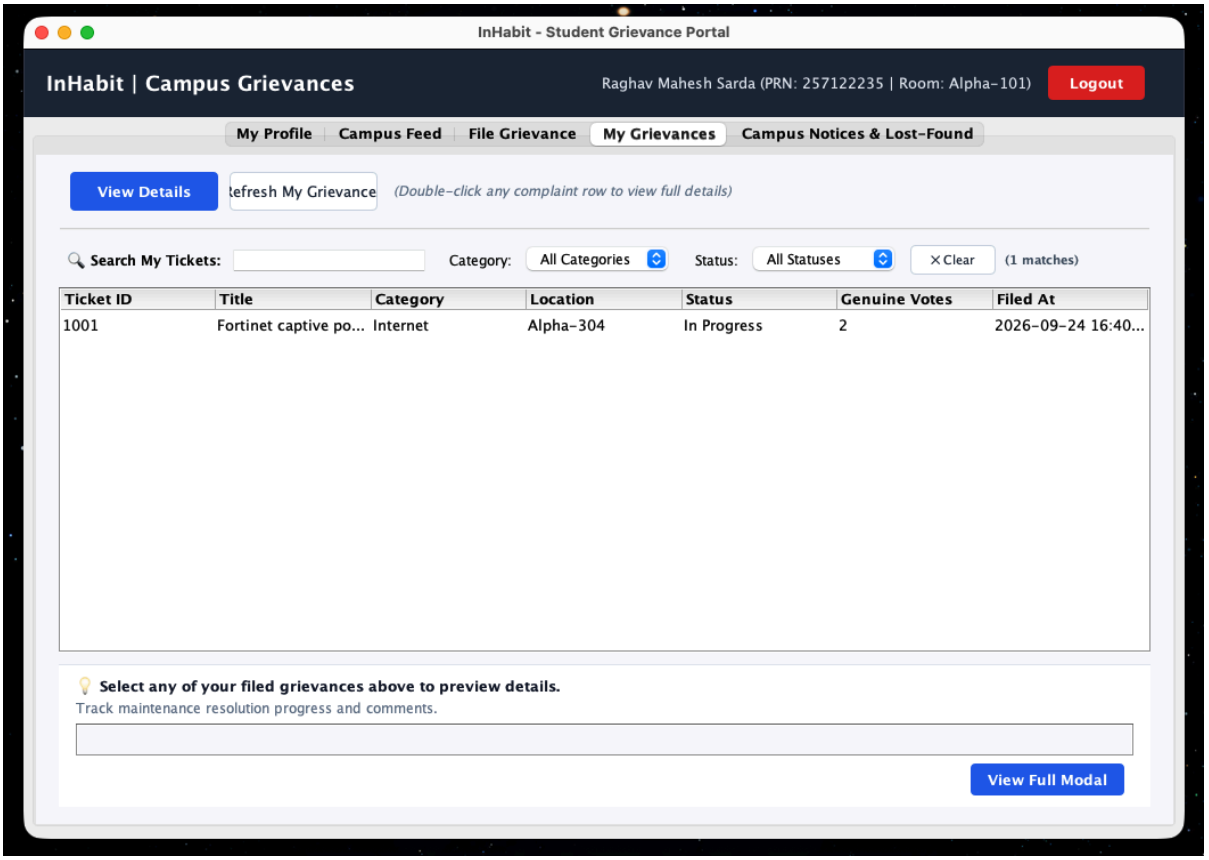
3. Campus Feed:

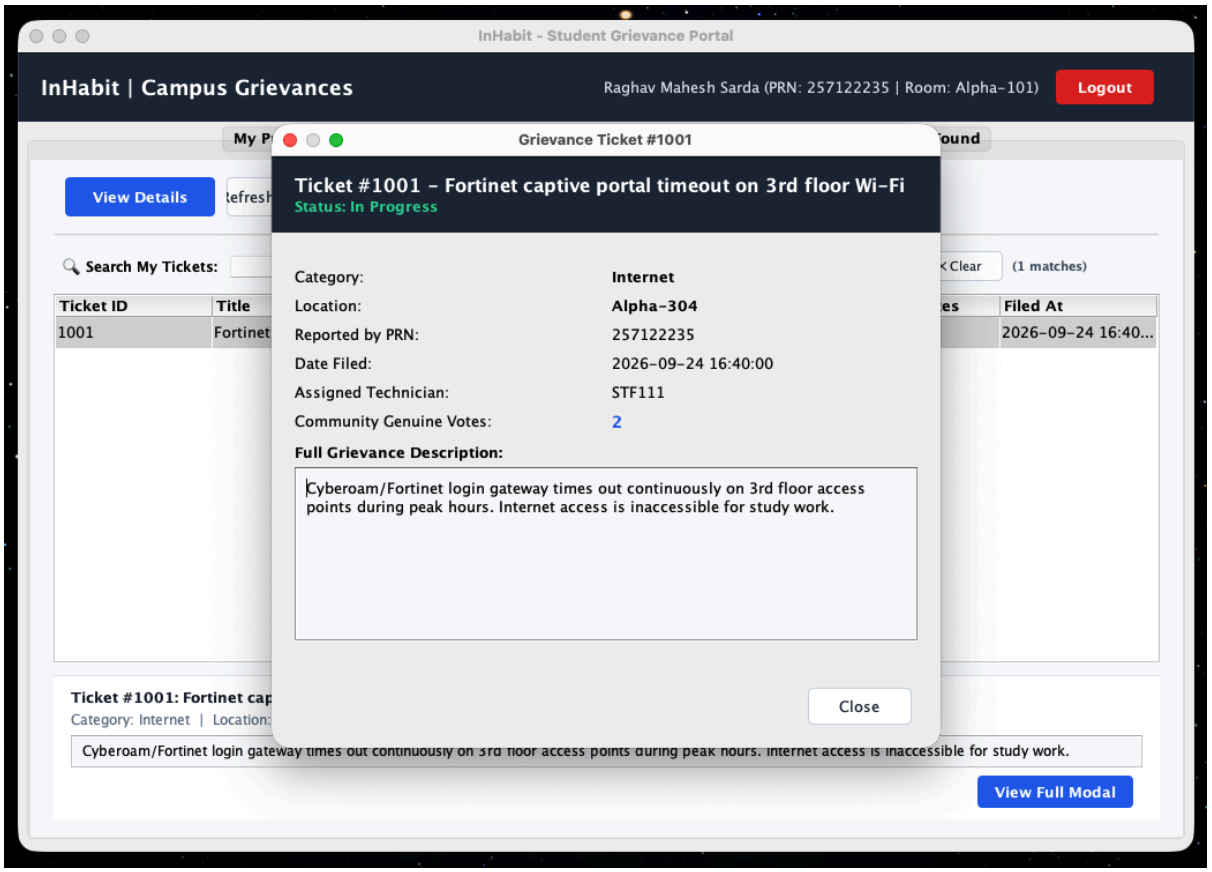




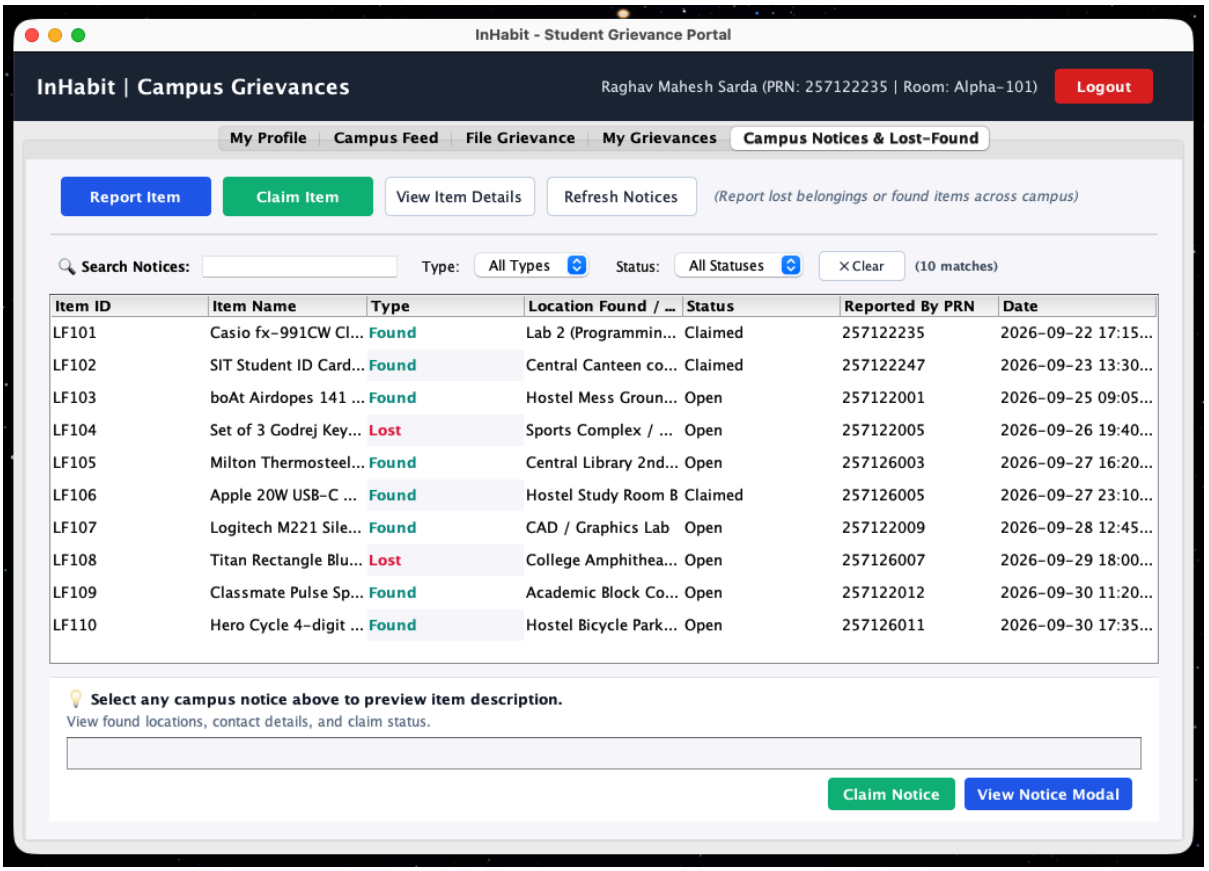
4. File Grievance & My Grievances:

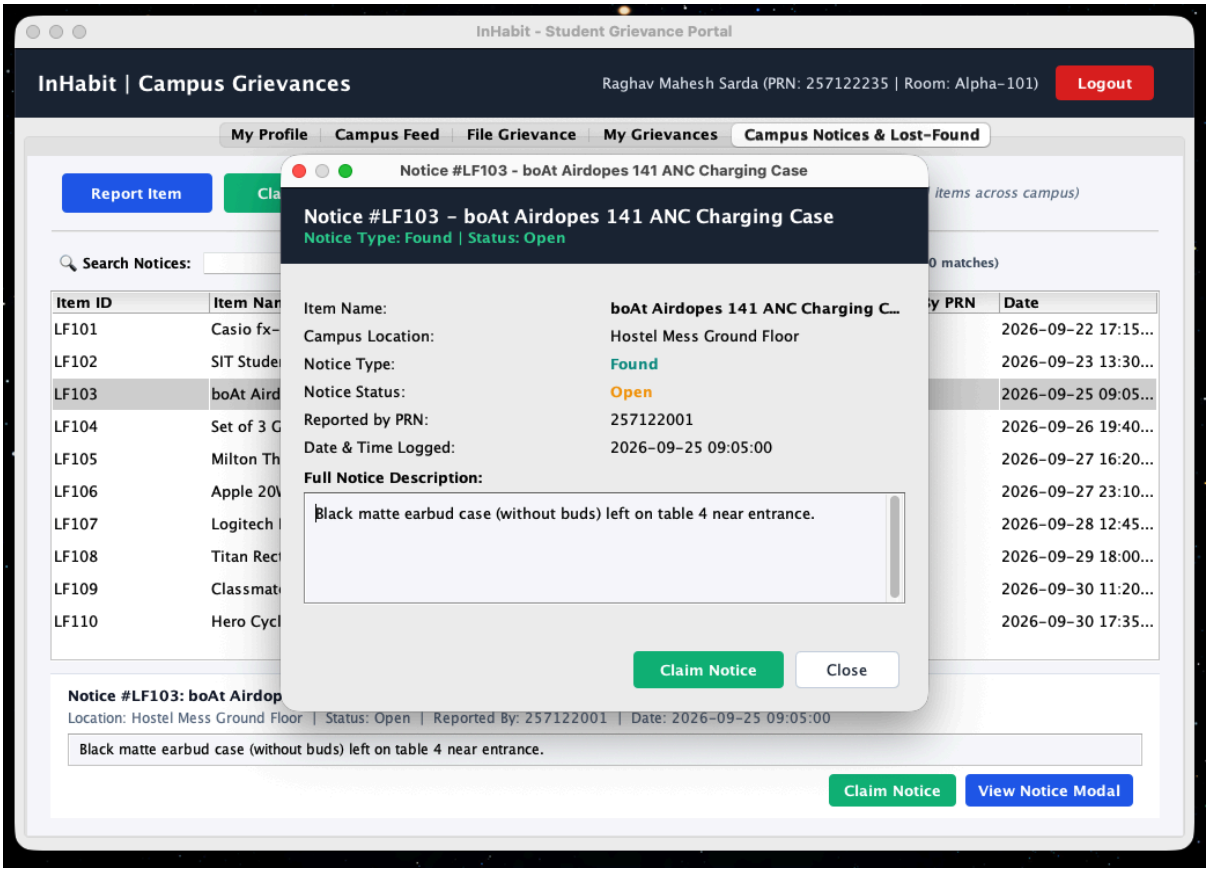





InHabit - Student Grievance Portal
InHabit | Campus Grievances
Raghav Mahesh Sarda (PRN: 257122235 | Room: Alpha-101)
Logout
View Details
Search My Tickets:
Clear
(1 matches)
Ticket ID
Title
Filed At
1001
2026-09-24 16:40...
Grievance Ticket #1001
Ticket #1001 – Fortinet captive portal timeout on 3rd floor Wi-Fi
Status: In Progress
Category: Internet
Location: Alpha-304
Reported by PRN: 257122235
Date Filed: 2026-09-24 16:40:00
Assigned Technician: STF111
Community Genuine Votes: 2
Full Grievance Description:
Cyberoam/Fortinet login gateway times out continuously on 3rd floor access points during peak hours. Internet access is inaccessible for study work.
Close
Ticket #1001: Fortinet cap
Category: Internet | Location:
Cyberoam/Fortinet login gateway times out continuously on 3rd floor access points during peak hours. Internet access is inaccessible for study work.
View Full Modal

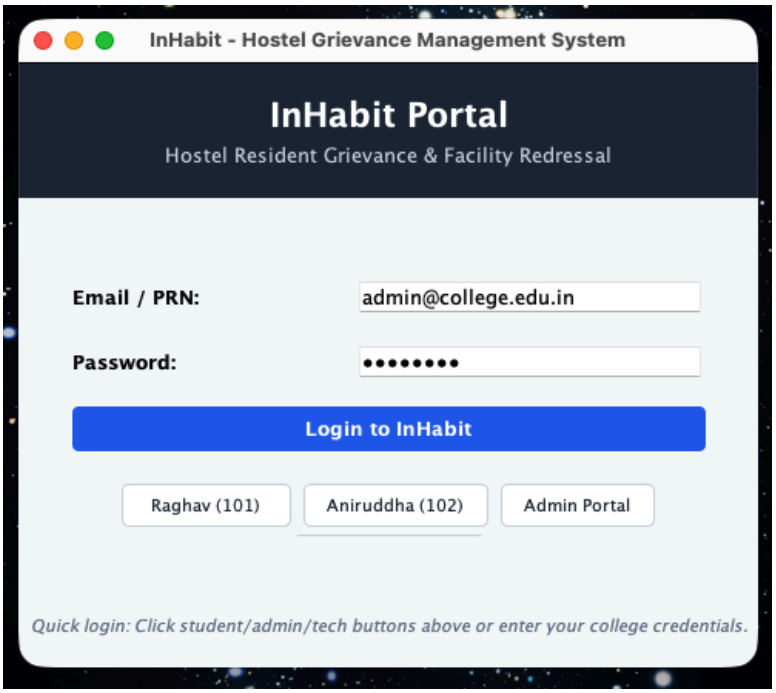
5. Campus Notices & Lost-Found:



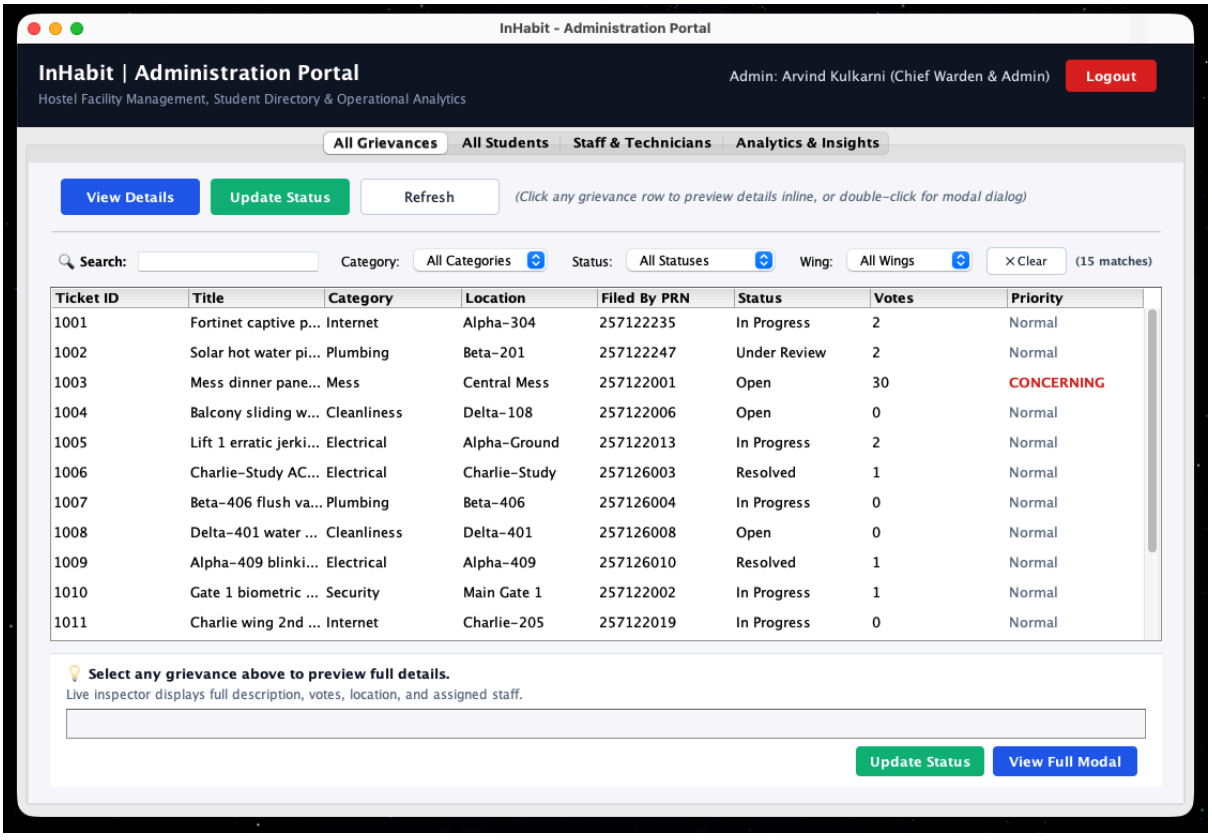


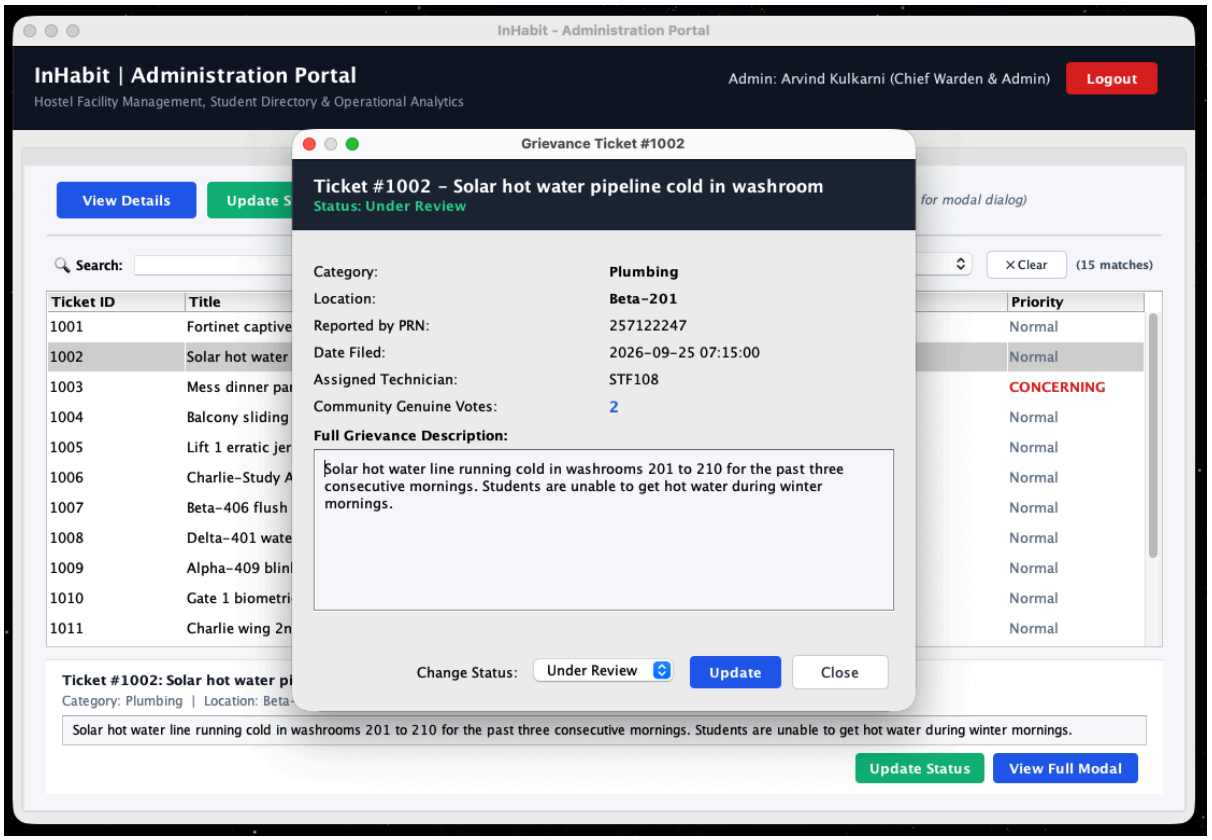
Admin:

1. Login Page:

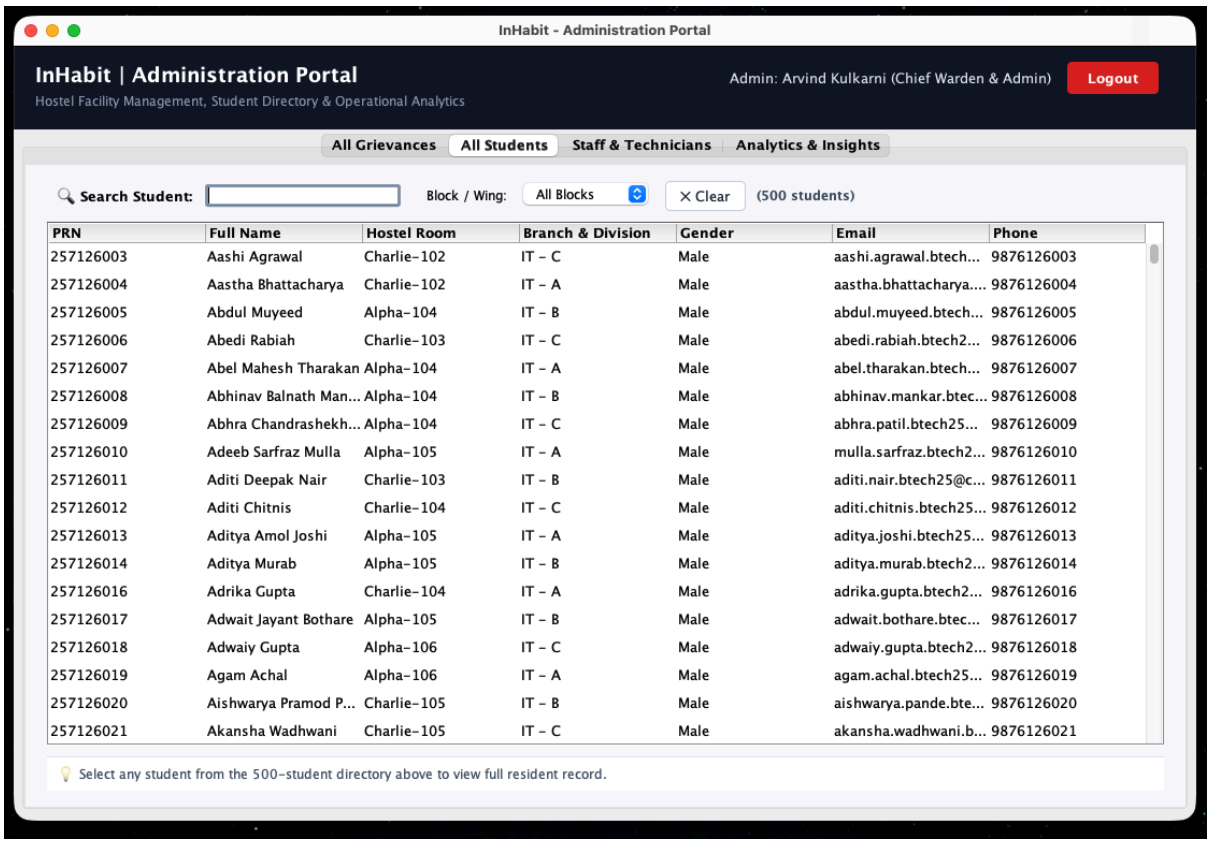


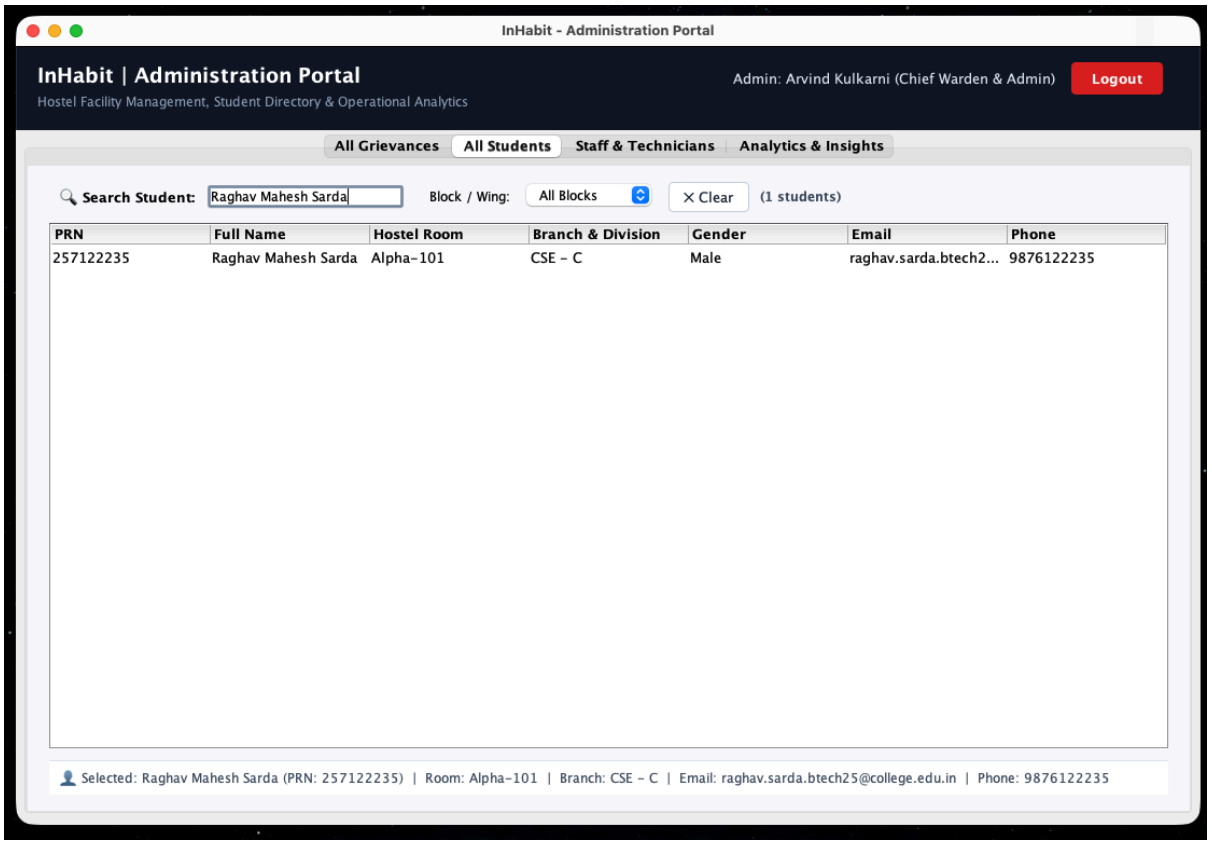
2. All Grievances:



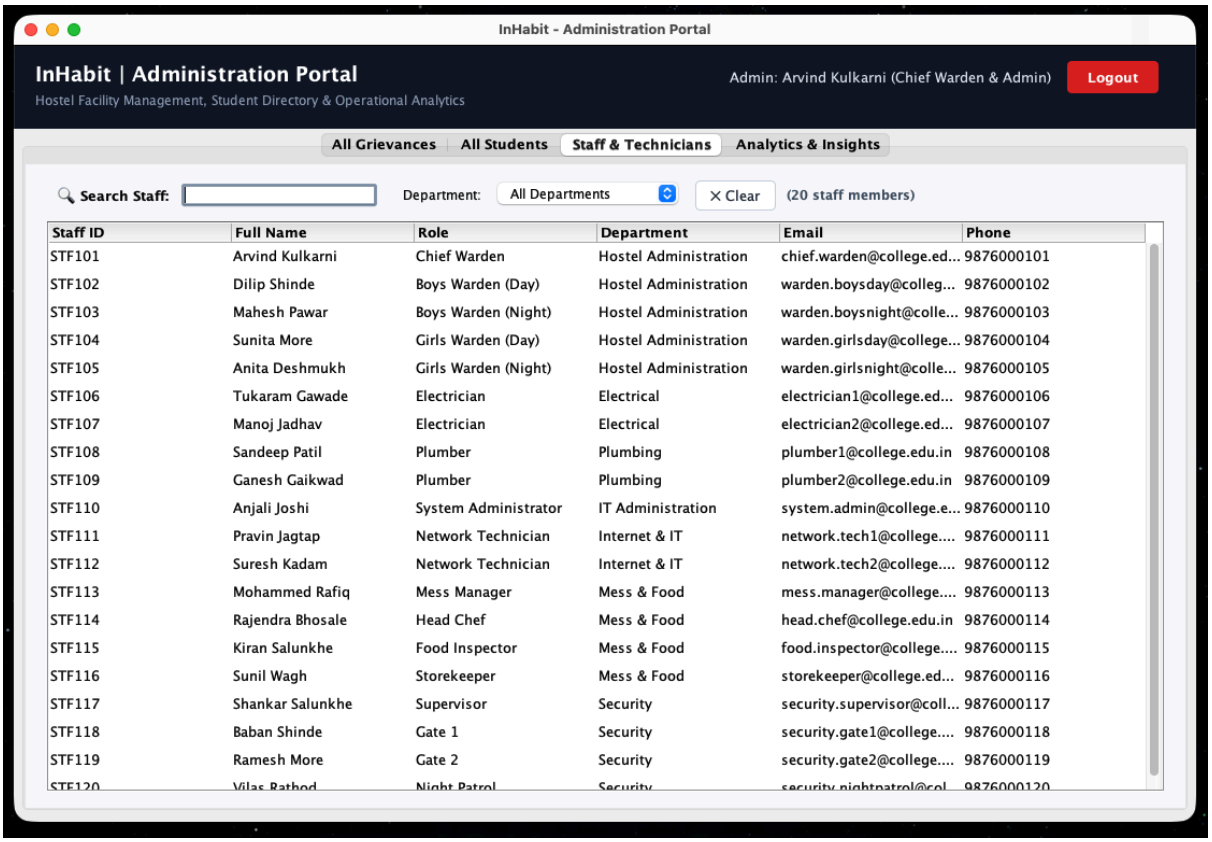


3. All Students:

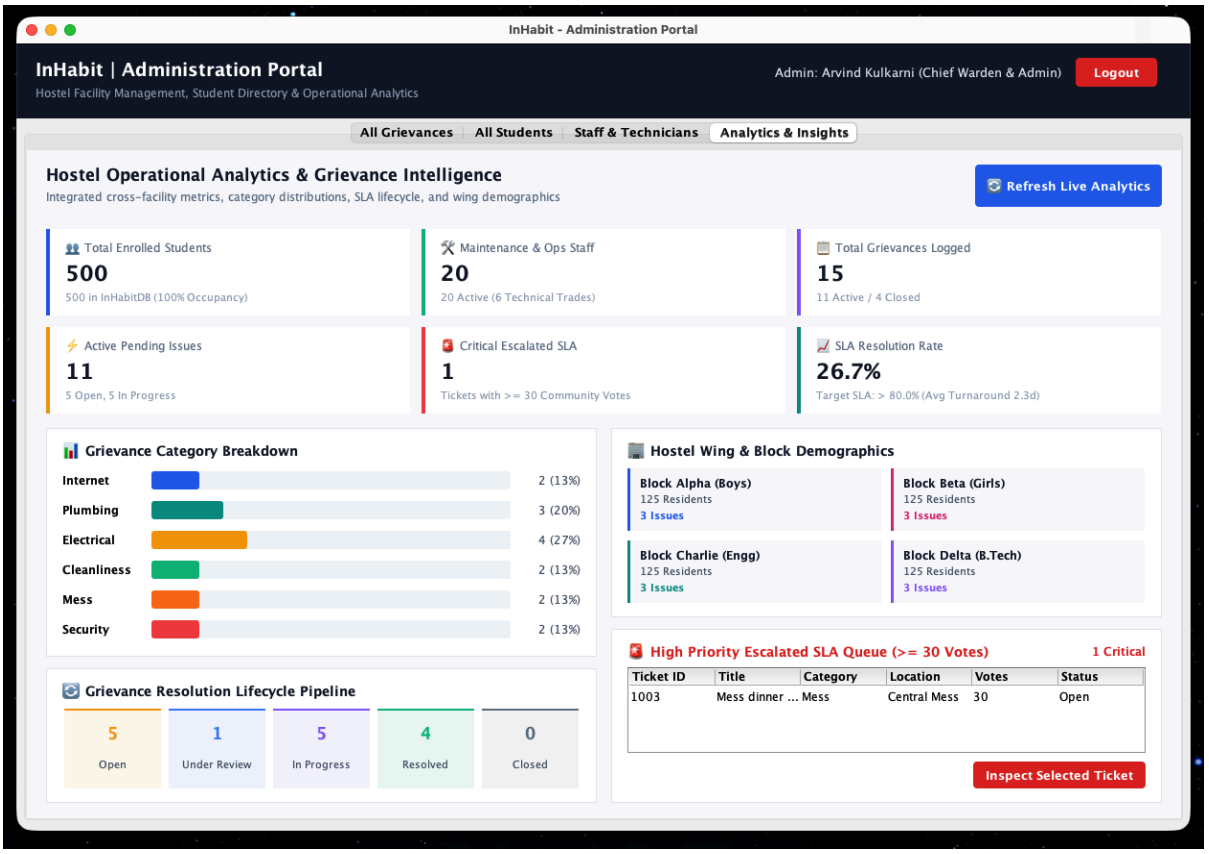




4. Staff & Technicians:



5. Analytics & Insights



---

13. References & Tools Used

Tools Used:

1. **Database Management System:** MySQL Community Server 8.0 & MySQL Workbench 8.0 CE
2. **Diagramming Software:** Draw.io for ER, EER, and Relational Schema Diagrams
3. **Application & Driver Stack:** Java SE (Swing/AWT) with MySQL Connector/J (mysql-connector-j)

References:

1. Silberschatz, A., Korth, H. F., & Sudarshan, S., *Database System Concepts*, 6th Edition, McGraw-Hill.
2. Elmasri, R., & Navathe, S. B., *Fundamentals of Database Systems*, 7th Edition, Pearson Education.

3. Ramakrishnan, R., & Gehrke, J., *Database Management Systems*, 3rd Edition, McGraw-Hill.
4. Codd, E. F., "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM*, 1970.

Contributing Members:

SR. NO.	NAME	PRN
1.	Raghav Sarda	25070122**235**
2.	Aniruddha Pratap Singh	25070122**247**
